

IntentChecker: An Intent-Model-Based Debugging Assistant for Mitigating Numeric Logic Errors in Solidity

Inseong Jeon¹, Sundeuk Kim¹, Hyunwoo Kim¹, Hoh Peter In^{1*}

^{1*}Department of Computer Science, Korea University, 145, Anam-ro, Seonbuk-gu, 02841, Seoul, Republic of Korea.

*Corresponding author(s). E-mail(s): hoh_in@korea.ac.kr;
Contributing authors: iwwyou@korea.ac.kr; sd_kim@korea.ac.kr;
khw0809@korea.ac.kr;

Abstract

Smart contract development lacks the mature debugging infrastructure available in traditional software engineering. In particular, numeric logic errors—where program behavior diverges from developer intent—are difficult to address. Whether such errors are present depends on the developer’s intended numeric behavior, which is not expressed in the code itself. Existing detection tools, designed to operate from source code alone in a testing context, rely on pre-defined vulnerability patterns and offer no mechanism for developers to declare and validate numeric intent during development. Errors not covered by these patterns therefore have no practical way of early prevention. This paper presents IntentChecker, a development-time intent validation tool for Solidity that operates at multi-contract single-transaction scope and shifts the detection target from pre-defined patterns to developer-defined intent. We propose an intent model with two types: @During for intended relationships at specific program points within a function, and @Post for intended relationships after function execution. A validation engine then checks whether the intervals computed by abstract interpretation satisfy the declared intent and reports a validation result together with a risk score. We evaluate IntentChecker on real-world numeric logic errors and show that it provides a detection signal complementary to existing pattern-based tools, with analysis latency suitable for interactive development.

Keywords: Solidity, Numeric Logic Error, Debugging Assistant, Intent Model, Intent Validation

1 Introduction

The debugging toolchain available to smart contract developers remains far less mature than what traditional software engineering provides, and the immutability of deployed contracts further amplifies the cost of bugs that go undetected (Soud et al., 2024). This immaturity is particularly acute for numeric logic errors—bugs in which a function executes without raising any compile-time or runtime exception but nevertheless produces values that violate the developer’s intended numerical behavior. Because correctness hinges on what the developer intended for a given computation—information that the source code does not capture—compilers, type checkers, and generic static analyzers are structurally blind to this class of bugs. A recent empirical study (Chaliasos et al., 2024) confirms this in practice: practitioners identified logic errors as the most difficult vulnerability type to detect, and reported that existing security tools are least effective against them.

Several recent tools—most notably NumScout (Chen et al., 2025) and GPTScan (Sun et al., 2024)—explicitly target numeric logic errors. These tools face an **inherent difficulty of the detection paradigm** rather than a mere engineering gap. First, without access to the developer’s actual intent, any detector must fall back on a curated catalog of codified patterns (e.g., division-in-path, operator order, wrong checkpoint order). Second, because these catalogs are necessarily incomplete, tool coverage is orthogonal: error patterns targeted by GPTScan are missed by NumScout, and vice versa, and no single tool can be expected to unify them. These are not failures of any specific tool but structural consequences of attempting to detect intent-dependent bugs without an explicit representation of that intent.

Compounding the above difficulty is a second, **temporal** limitation. Both NumScout and GPTScan are designed to run over completed code in auditing or testing contexts, after the function under inspection is fully written. At edit time, when the developer’s intent is still fresh in mind and iteration cost is lowest, there is currently no tool that lets the developer declare “I expect variable v to stay within range R at this point” and immediately see whether that intent is consistent with the abstract execution of the function being written. This edit-time gap is the central problem this paper addresses.

Our starting point for closing this gap is SolQDebug (Jeon et al., 2025), our previously published interactive debugger for Solidity. SolQDebug provides the missing piece of infrastructure: an interactive parser and an incremental control-flow-graph builder that together drive interval-domain abstract interpretation at statement-level granularity, accepting developer-supplied range annotations (@StateVar, @LocalVar) to inject symbolic inputs. However, SolQDebug only observes variable intervals; it does not validate any intended relationships between them, and its analysis scope is restricted to a single contract and a single transaction. SolQDebug’s own conclusion explicitly leaves numerical defects detection for future work. This paper realizes that future work.

We present IntentChecker, a development-time intent validation tool that builds on SolQDebug’s interactive analysis infrastructure and adds three contributions. First, we introduce an intent model—a formal specification language consisting of two annotation types that align with the stages of variable flow in a function body, together

with a denotational semantics that maps each clause to a tri-valued validation outcome against the abstract states produced by the interpreter. `@During` annotations specify intended relationships that must hold at specific program points within a function, while `@Post` annotations specify intended relationships that must hold after the function completes execution. Second, we develop a validation engine that maps each annotation clause to an interval-domain comparison and produces a graded signal of Satisfied, Violated, or Warning, together with a separately computed risk score on a 0–10 scale and the sub-intervals in which the intent may be violated. The underlying comparison primitive is mathematically proved sound against the set-theoretic specification of the intent model and cross-validated against brute-force ground truth across all possible topological configurations of two integer intervals. Third, we extend SolQDebug’s analysis scope from single-contract to multi-contract single-transaction, adding support for inherited contracts, library delegatecalls, interface-typed variables, and a practically useful arithmetic subset of inline assembly, and we introduce a new `@IReturn` debug annotation that injects range bindings into interface call return values, preserving the same “unknown value, developer-specified range” semantics already used by SolQDebug’s existing debug annotations.

To ground our motivation in concrete numbers, we analyzed 1,023 non-trivial functions from DAppSCAN-source (Zheng et al., 2024) (see Table 1) and found that numeric variable types appear in 82% of these functions, with `uint` alone used in 78%. Since numeric computations form the core of smart contract business logic—handling token transfers, fee calculations, and balance updates—explicitly specifying developer intent for these operations is both necessary and, as our evaluation shows, practical.

We evaluate IntentChecker on 89 real-world numeric logic errors drawn from Web3Bugs (Zhang et al., 2023) and NumScout (Chen et al., 2025). The evaluation organizes itself around three research questions: the extent to which IntentChecker can surface real-world numeric logic errors through its graded signal (RQ1), the intent model’s expressibility ceiling for cases that require bug awareness (RQ2), and how IntentChecker’s feedback compares with existing pattern-based detection tools in terms of signal characteristics, latency, and coverage complementarity (RQ3). Importantly, our positioning with respect to existing tools is complementary rather than competitive: IntentChecker fills a distinct place in the development workflow by enabling intent-driven validation during editing, and is intended to coexist with auditing-time detectors rather than replace them.

Contributions.

- We identify an unfilled gap in current Solidity tooling: the absence of a development-time mechanism for declaring and validating numeric intent, and we show that this gap is not a minor engineering oversight but a structural consequence of how existing numeric logic detection tools are designed (§2).
- We present an intent model with a formal denotational semantics defining each clause form as a tri-valued outcome against abstract states, together with a validation engine whose interval comparison primitive is proved sound against the semantics and cross-validated against brute-force ground truth, producing a graded signal (Satisfied / Warning / Violated) with a separately computed risk score and violated region (§4).

Table 1: Distribution of Variable Types in Smart Contract Functions

Category	Number of Functions	Percentage (%)
Total Functions Analyzed	1524	
Excluded Functions	501	
Functions Considered for Analysis	1023	
Functions using <code>uint</code>	798	78.00587
Functions using <code>int</code>	5	0.48876
Functions using <code>uint</code> and <code>int</code>	41	4.10557
Functions using other types	178	17.39980

- We extend SolQDebug’s single-contract single-transaction analysis scope to a multi-contract single-transaction scope that handles inheritance, libraries via delegatecall, interfaces, and a subset of inline assembly, and we introduce the `@IReturn` debug annotation to bridge the remaining cross-deployment gap (§3).
- We evaluate IntentChecker on 89 real-world numeric logic errors and position it against NumScout and GPTScan along three axes—signal characteristics, per-case latency, and coverage complementarity—demonstrating that the three tools occupy distinct points in the development/auditing timeline rather than forming a single comparison along detection accuracy (§5).

The remainder of this paper is organized as follows. Section 2 motivates the development-time intent validation problem and surveys the structural limitations of existing detection tools. Section 3 recaps SolQDebug and describes the scope extensions required for intent validation over multi-contract code. Section 4 presents the design of IntentChecker, including its intent model and validation engine. Section 5 presents evaluation results across the three research questions. Section 6 discusses the annotation authoring burden and remaining technical limitations. Section 7 reviews related work, and Section 8 concludes.

2 Background

2.1 Numeric Logic Errors in Solidity

Recent research has identified various numeric-related weaknesses in smart contracts (Chen et al., 2025; Sun et al., 2024; Soud et al., 2024; Zhang et al., 2023; Zhou et al., 2023). Building on these findings, numeric logic errors can be defined as follows:

Definition 1 (Numeric Logic Error) Bugs that allow transactions to complete successfully without compile-time or runtime exceptions, but violate the developer’s intended numerical or state relationships during function execution.

These errors can manifest in different execution contexts. Single-transaction errors occur within the execution of a single function call—where computation, function

calls, and output generation happen in one atomic sequence. In contrast, multiple-transaction errors only emerge through sequences of function calls, requiring analysis of inter-function dependencies and contract-level invariants. Examples of multiple-transaction errors include Risky First Deposit and Price Oracle Manipulation (Sun et al., 2024; Zhang et al., 2023), where attackers exploit state changes across multiple function invocations.

This paper focuses on single-transaction function execution, as this aligns with the development-phase validation model where developers can immediately verify individual function behaviors against their intentions. The representative single-transaction numeric logic error patterns identified in prior work are summarized below:

- **Div In Path** (Chen et al., 2025): Division results used in conditions can lead to unexpected control flow due to integer truncation.
- **Operator Order Issue** (Chen et al., 2025): Arithmetic order matters: $a/b*c \neq a*c/b$ due to intermediate truncation.
- **Exchange Problem** (Chen et al., 2025): Rounding errors in exchange calculations may result in zero output despite valid input, or receiving tokens without payment.
- **Precision Loss Trend** (Chen et al., 2025): Integer division’s inherent truncation tends to favor one side, potentially misaligning with protocol intent.
- **Minor Amount Retention** (Chen et al., 2025): Integer division in distributions leaves remainders unallocated to any recipient, causing permanent lock.
- **Erroneous Accounting** (Sun et al., 2024; Zhang et al., 2023): Incorrect implementation of business model formulas introduces errors that accumulate into substantial losses. Representative sub-patterns include:
 - Wrong Interest Rate Order: Balance calculations using outdated or prematurely updated interest rates yield incorrect results.
 - Wrong Checkpoint Order: Incorrect ordering between state updates and checkpoints leads to reward miscalculation.
- **Inconsistent State Updates** (Zhang et al., 2023): A required state variable update is omitted, causing subsequent calculations to use stale values. For example, failing to accrue interest before a balance query produces incorrect results.
- **Greedy Contract** (Soud et al., 2024): Contract can receive funds but lacks proper logic to withdraw them, causing permanent lock.

Despite their diversity, these error patterns share a common root: the actual behavior diverges from what the developer intended. A straightforward consequence is that any tool that hopes to flag such bugs reliably must reason about the intended behavior, not only the written code. The next subsection examines how existing tools handle—or rather, fail to handle—this requirement.

2.2 Limitations of Current Work

Several tools explicitly target numeric logic errors. NumScout (Chen et al., 2025) catalogs six numerical defect patterns (Div In Path, Operator Order Issue, Exchange Problem, Precision Loss Trend, Minor Amount Retention, and Profit Opportunity) and detects them through LLM-pruned symbolic execution. GPTScan (Sun

et al., 2024) addresses a different group of logic vulnerabilities, including several accounting-related patterns, by combining GPT-based scenario matching with lightweight program analysis. Other works (Zhang, 2024; Wang et al., 2024b) take closely related approaches. These tools represent the current state of the art and have meaningfully advanced the field. However, when viewed through the lens of the problem defined in §2.1, they share two structural limitations that are not engineering oversights but rather direct consequences of their design.

(1) Operation context: post-hoc auditing rather than development. Existing numeric logic error detection tools are designed to run over completed code, and are positioned within auditing or testing workflows. They accept a contract (or a set of contracts) and emit findings as a report. They do not run inside the developer’s editor as code is being written, nor do they accept developer-supplied specifications about the intended behavior of the function currently under the cursor. As a consequence, the developer’s intent is not part of the input at all. Any detector operating in this context must recover intent indirectly: either by hard-coding intent assumptions as “patterns” that describe what buggy code looks like (NumScout), or by asking a language model to infer intent from code, comments, and public documentation (GPTScan, SmartInv (Wang et al., 2024b)). Both strategies work only to the extent that the assumed intent coincides with the actual developer intent in the specific contract being analyzed.

(2) Approach: pattern-based coverage is orthogonal across tools. Because the operation context rules out direct access to intent, each tool must commit to a fixed catalog of patterns. These catalogs are necessarily incomplete: NumScout’s six numerical defect patterns and GPTScan’s accounting-related patterns target disjoint regions of the bug space. A bug matching a NumScout pattern is, in general, not matched by GPTScan, and vice versa. The problem is not that one catalog is better than the other; it is that neither catalog can be extended to cover all intent-dependent bugs, because intent itself is not expressible as a static pattern over code. This orthogonality means that “adding another pattern-based tool” cannot in principle converge on full coverage.

These two limitations are structural, not tool-specific. We emphasize that we do not characterize the above limitations as failures of NumScout or GPTScan. Both tools are well-engineered within the design space they chose. The point is that the design space itself—post-hoc pattern-based detection without explicit intent input—has an upper bound, and the field has largely reached it. To go further, one must either enlarge the input (ask the developer to supply their intent) or move the tool earlier in time (run it during editing, when the developer is still available to supply that intent). The approach taken in this paper is to do both.

2.3 Proposed Overview

The structural limitations above suggest a complementary direction rather than a competing one. If existing detectors cannot see the developer’s intent, a tool that sits in the editor and explicitly asks the developer to declare their intent can fill the remaining

space without contradicting the existing tools’ strengths. Our prior work, SolQDe-
bug (Jeon et al., 2025), already provides much of the infrastructure needed for such
a tool: an interactive parser, an incremental control-flow graph, and interval-domain
abstract interpretation that accepts developer-supplied range annotations (@StateVar,
@LocalVar) and produces line-level feedback as code is being written. What SolQDe-
bug does not do is validate any intended relationships between the computed intervals,
and its analysis scope is confined to a single contract and a single transaction.

IntentChecker takes SolQDebug as its foundation and adds three things. First,
it introduces an intent model—two annotation types, @During and @Post, together
with a grammar of clauses that lets developers declare what numeric relationships
should hold at specific program points and after function execution, and a denotational
semantics that assigns each clause a tri-valued validation outcome against the abstract
states produced by the interpreter. Second, it introduces a validation engine that
reads the intervals computed by the extended abstract interpreter, compares them
against the declared intent, and produces a graded result: Satisfied when the intent
holds for all values in the intervals, Violated when it fails for all values, and Warning
when it holds for some and fails for others. Each result is accompanied by a risk
score on a 0–10 scale and, for non-Satisfied results, by the sub-interval in which the
intent may be violated. Third, it extends the analysis scope to cover multi-contract
single-transaction code: inherited contracts, libraries invoked via using directives and
delegatecall, interface-typed variables, and a practically useful arithmetic subset of
inline assembly. To handle return values from interface calls whose implementation
code is unavailable, we introduce @IReturn, a new debug annotation that uses the
same “inject a range at an unknown point” semantics already used by SolQDebug’s
existing @StateVar and @LocalVar annotations.

Given an intent clause $l \text{ op } r$ where l ranges over interval L and r ranges over
interval R , the validation result is deterministically categorized into one of three states:

Satisfied $\iff \forall l \in L, \forall r \in R : l \text{ op } r \text{ holds}$

Violated $\iff \forall l \in L, \forall r \in R : l \text{ op } r \text{ fails}$

Warning $\iff$ neither Satisfied nor Violated

Unlike warnings from pattern-based static analyzers, which are heuristic indicators
that a known buggy shape may be present, IntentChecker’s Warning is a precise
statement: the declared intent holds for some combinations of values in L and R but
not others, and the tool reports exactly which sub-intervals fall into the violating
region. In this sense IntentChecker is best understood as an awareness tool rather than
a detector: its goal is not to pronounce “this is a bug” but to bring the developer’s
attention to the specific program points where the abstract execution cannot confirm
the declared intent. The next section describes the SolQDebug foundation and the
scope extensions in detail, and Section 4 presents the intent annotation grammar and
the validation engine.

3 Preliminaries: SolQDebug Foundations and Scope Extension

This section reviews the parts of SolQDebug (Jeon et al., 2025) that IntentChecker relies on and describes the scope extensions we introduce to support intent validation over realistic multi-contract Solidity code. Readers familiar with SolQDebug may skip §3.1 and proceed directly to §3.2.

3.1 SolQDebug Recap

SolQDebug is an edit-time debugging assistant for Solidity that provides statement-level feedback through interval-domain abstract interpretation. It addresses two forms of inefficiency in traditional Solidity debugging: the temporal cost of the compile–deploy–setup–trace cycle in tools such as Remix IDE, and the iterative cost imposed by the non-reversible state model of the EVM. Both inefficiencies are resolved by performing the entire analysis inside the editor, without contacting any blockchain node.

Interactive parsing and incremental CFG. SolQDebug extends the Solidity ANTLR grammar with seven specialized entry rules that accept partial code fragments—a lone else branch, a function header without a body, a single statement inside an existing block, and so on. As the developer types or edits code, only the affected region is re-parsed, and the corresponding nodes are inserted into an incremental control-flow graph via a line-to-node index. Abstract interpretation then resumes from the edit point, reusing previously computed intervals for nodes whose inputs have not changed.

Debug annotations and the “unknown value, injected range” semantics. Because abstract interpretation over completely unconstrained inputs produces overly conservative results, SolQDebug lets developers supply range information for variables whose actual values are unknown to the analyzer. Two debug annotation families cover this role:

- **@StateVar**—for global and state variables whose values are determined by prior transactions (e.g., `totalSupply`, a user’s balance, a lending pool’s reserves). SolQDebug has no way to know what value these variables hold at the start of the function being debugged; the developer declares a representative range instead.
- **@LocalVar**—for function parameters and locally declared variables whose values are determined by the caller or by earlier computation. This is how the developer specifies the test cases under which the function should be analyzed.

Both annotations share a single, unifying semantics: at each program point where the analyzer would otherwise have only $\top$ (“any value”), the developer explicitly injects a bounded range. The analyzer then proceeds to compute intervals for downstream program points under that injected range, giving the developer a line-by-line view of how the function behaves for the chosen inputs.

Output semantics: range in, range out. SolQDebug’s output for each annotated path is a table of intervals—one per variable per line—reflecting what the abstract

execution computed. SolQDebug itself makes no judgment about whether these intervals are “correct”: it neither compares them against any specification nor flags any of them as buggy. It is, in the strict sense, a debugger: it shows the developer what happens, and leaves the interpretation to the developer. IntentChecker’s validation engine is precisely the layer that adds such interpretation on top of this “range in, range out” foundation.

Analysis scope. SolQDebug’s published scope is deliberately narrow: single contract, single transaction. A function in the target contract is analyzed under the assumption that its state variables hold the values declared by `@StateVar`, and that only that function executes. Inter-contract calls, inheritance, and library invocations are explicitly out of scope and are left to future work in the SolQDebug paper. The next subsection describes how IntentChecker extends this scope to handle the realistic multi-contract code that real numeric logic errors tend to live in.

3.2 Scope Extension for Multi-Contract Single-Transaction Analysis

Real-world numeric logic errors rarely occur in single-contract code: most of the Web3Bugs and NumScout benchmark cases involve inherited contracts, library calls, or interface-typed variables. Validating intent over such code requires extending SolQDebug’s analysis scope from single contract to multi-contract single-transaction, where a single atomic transaction may touch several contracts located in the same source file or its immediate dependencies, but is still confined to one execution step. This subsection describes the four extensions required to make this scope practical: (a) a deployment-unit model for identifying which contracts share storage with the target, (b) library and using-directive handling, (c) interface-typed variables and the `@IReturn` debug annotation, and (d) a practically useful subset of inline assembly.

(a) Deployment-unit scope. IntentChecker treats as “in-scope” everything that shares storage with the target contract in a single EVM execution: the target contract itself, all inherited parents resolved via C3 linearization, and any libraries invoked through `delegatecall` via Solidity’s `using` for directive. These all run under the same `address(this)` and over the same storage, so debug annotations on the target contract’s state variables can be reused without reinterpretation whenever the analyzer steps into an inherited function or a library function. External contracts—i.e., separately deployed contracts invoked through an address or an interface-typed variable—are out-of-scope by default, because their storage is a different deployment unit that SolQDebug’s debug annotations cannot reach. Extension (c) below describes how `@IReturn` allows a restricted form of data injection across this deployment boundary for view and pure interface calls.

(b) Library and interface handling. For each Solidity construct in the target source file, IntentChecker pre-builds a dedicated CFG: contracts are stored as `ContractCFG`, libraries as `LibraryCFG`, and interfaces as `ContractCFG` instances marked with a dedicated flag whose functions consist of an empty entry-to-exit body (interfaces carry no implementation). When a `using LibraryName for T` directive is encountered,

the corresponding LibraryCFG is attached to the declaring contract’s set of using-libraries, and subsequent method-call resolution on values of type T walks this set to find the matching library function overload. Inherited functions are resolved via the same C3 linearization order used by the Solidity compiler, ensuring that overrides behave as they would at runtime. Library function bodies are analyzed in-place, reusing the debug annotations of the target contract since a library invoked via `delegatecall` shares storage with its caller.

(c) @IReturn: Interface Return Value Debug Annotation. Interface-typed variables (e.g., `IOracle oracle`) present a structural problem that debug annotations on state or local variables cannot solve: the implementation code behind the interface is not present in the source file at all, so SolQDebug’s abstract interpreter must return $\top$ for any value derived from an interface call. This $\top$ then propagates into every downstream computation, defeating the purpose of intent validation for any function that reads an oracle or queries an external token contract.

Our observation is that this situation is semantically identical to the case that `@StateVar` and `@LocalVar` already handle. In all three cases, the analyzer encounters a program point at which the actual value is unknown because its origin lies outside the analysis scope: `@StateVar` handles unknown storage values left behind by prior transactions, `@LocalVar` handles unknown inputs supplied by the caller, and a fourth kind of unknown—the return value of an interface call—can be handled by the same “inject a developer-specified range at the unknown point” mechanism. This is consistent with the underlying philosophy of debugging: the developer chooses a particular execution path to inspect, declares the inputs relevant to that path, and observes the line-by-line output. An interface return value is simply another input to the path under analysis; it only happens to originate from beyond a deployment-unit boundary rather than from a local variable or a storage slot.

We formalize this intuition as `@IReturn`, a new debug annotation that lets the developer bind a range to the return value of a specific interface function. Two surface syntaxes are supported:

- **Pattern A (variable-qualified).** `// @IReturn oracle.getPrice() = [1000, 2000]` binds the return value of `getPrice` on an interface-typed variable (oracle must be declared as some interface type).
- **Pattern B (explicit cast).** `// @IReturn IOracle(oracleAddr).getPrice() = [1000, 2000]` binds the return value of `getPrice` when the call is written as an explicit cast of an address variable to an interface type.

Both patterns may target structured return values, including struct members (`...getReserves().reserve0`) and tuple elements (`...getPair()[0]`), which we represent as an access chain so that a single return value can be partially specified and partially left as $\top$.

View/pure restriction. `@IReturn` is restricted to view and pure interface functions. The reason is that a state-modifying interface call would, in general, alter the state of the external contract whose implementation we do not have, and `IntentChecker` has no principled way to model the resulting storage mutation in the external deployment unit. Allowing `@IReturn` on state-modifying calls would let the developer specify the

Algorithm 1 ProcessIReturn: Annotation Registration

```
1: Input: annotation contractVar.func().chain = value
2: Output: updated IReturn registry  $\mathcal{R}$ , or error

3:  $I \leftarrow$  resolve contractVar to its declared interface type
4: if  $I$  is undefined then return Error(“not an interface variable”)
5: end if
6:  $f \leftarrow$  look up func in interface  $I$ 
7: if  $f$  is undefined then return Error(“function not in interface”)
8: end if
9: if  $f$  is not view or pure then
10:   return Error(“@IReturn restricted to view/pure”)
11: end if
12:  $\mathcal{R}[(contractVar, func, chain)] \leftarrow value$ 
```

return value but silently omit any side effect, producing a model that does not correspond to any real execution. Restricting @IReturn to view/pure functions avoids this unsoundness: by definition, a view or pure call has no side effects, so binding its return value is a complete description of its observable behavior.

Registration. Algorithm 1 describes how an @IReturn annotation is validated and registered. Like @StateVar, @LocalVar, and @GlobalVar, @IReturn annotations are processed as part of the batch annotation phase: the developer writes all debug annotations, and the analyzer then interprets the target function from its start block with all registered values in effect. The key difference is the view/pure check (line 4): if the target function is state-modifying, registration is rejected with a diagnostic, preventing the unsound model described above.

Resolution at call sites. When the abstract interpreter encounters an interface function call during analysis, it must decide whether to return $\top$ (no information) or a developer-specified value. Algorithm 2 describes the resolution procedure. The registered entries from Algorithm 1 are looked up as a short-circuit check before the interpreter would otherwise default to $\top$. Two surface patterns are tried in order: Pattern A matches by the variable name that holds the interface reference (e.g., `oracle.getPrice()`), and Pattern B matches by the explicit interface cast (e.g., `IOracle(addr).getPrice()`). If a match is found, the registered value is assembled—potentially following an access chain for struct members or tuple indices—and returned immediately, preventing $\top$ from propagating into downstream computations.

(d) Arithmetic-focused inline assembly support. Solidity’s inline assembly (assembly { ... }) is written in Yul and frequently appears in arithmetic-heavy code, particularly in high-precision fixed-point libraries such as FullMath or ABDKMath. SolQDebug originally treated any inline assembly block as a single opaque region, leaving all affected variables at $\top$. IntentChecker implements a dedicated Yul visitor that recognizes the subset of Yul used for arithmetic computation and translates it into the same expression IR used by the rest of the analyzer. Concretely, the visitor supports variable declarations (`let x := expr`) and assignments (`var := expr`) inside a

Algorithm 2 ResolveIReturn: Interface Call Value Resolution

```
1: Input: interface call expression  $e = base.func(\dots)$ , IReturn registry  $\mathcal{R}$ 
2: Output: resolved value or  $\top$ 

    // Pattern A: variable-qualified (e.g., oracle.getPrice())
3: if  $base$  is an identifier with interface type  $I$  then
4:    $entries \leftarrow \{c \mapsto v \mid (base, func, c) \mapsto v \in \mathcal{R}\}$ 
5:   if  $entries \neq \emptyset$  then
6:     return ASSEMBLE( $I, func, entries$ )
7:   end if
8: end if

    // Pattern B: explicit cast (e.g., IERC20(addr).balanceOf())
9: if  $base$  is a cast  $I(addr)$  then
10:   $entries \leftarrow \{c \mapsto v \mid (I, addr, func, c) \mapsto v \in \mathcal{R}\}$ 
11:  if  $entries \neq \emptyset$  then
12:    return ASSEMBLE( $I, func, entries$ )
13:  end if
14: end if

15: return  $\top$  ▷ no registry match; fall back to default

16: function ASSEMBLE( $I, func, entries$ )
17:   if  $entries$  contains only the empty chain  $()$  then
18:     return  $entries[()]$  ▷ single scalar return
19:   end if
20:    $retType \leftarrow$  load return type schema of  $I.func$  from interface CFG
21:    $val \leftarrow \top_{retType}$  ▷ initialize with  $\top$  of the declared type
22:   for all  $(c, v) \in entries$  do
23:     set  $val$  at access chain  $c$  to  $v$  ▷ struct member or tuple index
24:   end for
25:   return  $val$ 
26: end function
```

yulBlock, and maps the following Yul built-ins to their Solidity operator counterparts: arithmetic add, sub, mul, div/sdiv, mod/smod, exp; comparison lt/gt/slt/sgt/eq; bit-wise and, or, xor; shifts shl, shr, sar; the unary not and iszero; and the three-argument mulmod and addmod, which are expanded into the corresponding Solidity expressions $(a \cdot b) \bmod n$ and $(a + b) \bmod n$. This coverage is deliberately chosen to capture the arithmetic that high-precision libraries typically express in Yul to bypass Solidity's overflow checks, while avoiding the control-flow and memory features (if, for, switch, mload/mstore, sload/sstore) whose abstract semantics would require separate engineering beyond the scope of this paper. Yul built-ins outside the supported subset (e.g., keccak256, call, log0) are handled conservatively by leaving their results at $\top$,

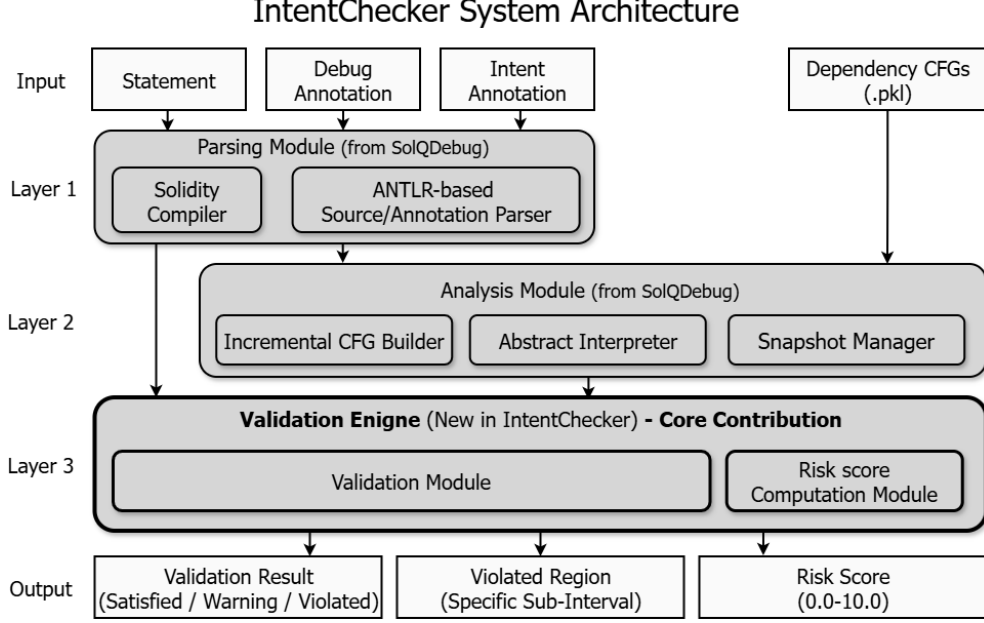


Fig. 1: System architecture of IntentChecker. Layers 1–2 (Parsing and Analysis Modules) are reused from SolQDebug; Layer 3 (Validation Module) is the core contribution of this work. The Validation Engine receives parsed intent clauses from the Parsing Module and computed intervals from the Analysis Module.

preserving soundness. Section 6 discusses why we chose this particular subset and what the remaining unsupported features mean for the analysis.

Summary. Taken together, these four extensions transform SolQDebug’s single-contract single-transaction sandbox into an analysis scope that covers the ways modern Solidity contracts are actually structured: multiple contracts per source file, library-mediated arithmetic, oracle-style interface calls, and hand-written Yul for performance-critical fixed-point math. All extensions preserve SolQDebug’s “range in, range out” character; none of them introduce a notion of correctness on their own. Building the intent validation layer on top of this extended analyzer is the subject of the next section.

4 The Design of IntentChecker

4.1 System Architecture

Figure 1 illustrates the overall architecture of IntentChecker.

Input. IntentChecker receives three types of input:

- **Statement**—individual Solidity statements provided incrementally as the developer writes code.

- **Intent Annotation**—@During and @Post specifications expressing the developer’s intended numerical relationships.
- **Debug Annotation**—variable initialization directives that provide concrete or symbolic value ranges at program points where the actual value is unknown to the analyzer. This category includes SolQDebug’s original @GlobalVar (constructor-initialized storage variables), @StateVar (storage variables modified at runtime), and @LocalVar (function parameters and local variables), as well as the @IReturn annotation introduced in §3.2(c) for interface call return values.

Output. For each intent annotation, IntentChecker produces:

- **Result**—a validation state of Satisfied, Violated, or Warning.
- **Risk Score**—a 0–10 scale score: 0.0 for Satisfied, 10.0 for Violated, and 0.1–9.9 for Warning reflecting the severity of the potential violation.
- **Violated Region**—the specific sub-intervals where the intent may be violated, helping developers identify problematic input ranges.

Internal Modules. The system builds upon SolQDebug (Jeon et al., 2025), whose Parsing Module, Analysis Module, and Snapshot Manager are reused as the basis for IntentChecker’s extended analysis. The Parsing Module uses ANTLR for syntax analysis of both Solidity source fragments and intent/debug annotation comments, and delegates to the official Solidity compiler for semantic validation. The Analysis Module performs incremental abstract interpretation over the interval domain, updating the control-flow graph and variable states as each statement is processed, and handles the multi-contract scope extensions (inheritance, library resolution via using, interface-typed variables, and the arithmetic Yul subset) described in §3.2. For every library and interface referenced from the target source file, the Analysis Module either loads a pre-analyzed Dependency CFG—a serialized LibraryCFG or ContractCFG object produced once and cached—or constructs one on-the-fly when the source is locally available. When a debug annotation is received, the Snapshot Manager preserves the current analysis state, allowing the system to apply temporary variable bindings, re-analyze affected code paths, and restore the original state afterwards; this mechanism is reused without modification for the new @IReturn annotation, since injecting a range at an interface call return site is mechanically the same as injecting one at a @StateVar declaration.

The Validation Engine (Layer 3) is IntentChecker’s core contribution on top of this extended analyzer. It receives two inputs from the lower layers: parsed intent clauses from the Parsing Module, which encode the relational constraints declared in @During/@Post annotations, and computed variable intervals from the Analysis Module, which represent the abstract state of each variable at the annotated program point. For each clause, it evaluates whether the declared intent holds by comparing the actual intervals against the specified constraints, producing the validation result, risk score, and violated regions as described above.

4.2 Motivating Examples

This section illustrates how IntentChecker addresses representative numeric logic errors, using three real-world cases from our evaluation benchmark (§5). The first two cases demonstrate mid-execution checks via @During annotations; the third demonstrates a post-execution check via @Post.

4.2.1 Operator Order Issue (BoostToken)

Listing 1 shows a fee-distribution function that divides before multiplying, causing integer truncation to silently reduce the transferred amount. The developer intends the marketing wallet to receive $\lfloor \text{amount} \times 5/12 \rfloor$, but `amount.div(12).mul(5)` truncates $\text{amount}/12$ first, losing precision when `amount` is not a multiple of 12.

Listing 1: BoostToken: operator order issue in fee distribution (simplified).

```
1 function sendETHToTeam(uint256 amount) private {
2     // @LocalVar amount = [68, 68]
3     _devWalletAddress.transfer(amount.div(4));
4     // @During transfer.arg[0] >= amount * 5 / 12
5     _marketingWalletAddress.transfer(amount.div(12).mul(5));
6     // @During transfer.arg[0] >= amount * 2 / 9
7     _dipWalletAddress.transfer(amount.div(9).mul(2));
8     _marketingWalletAddress2.transfer(amount.div(9));
9 }
```

The @During `transfer.arg[0] >= amount * 5 / 12` annotation expresses the developer's intent that the actual transfer argument should be at least the mathematically correct share. With `amount = 68`, the correct value is $\lfloor 68 \times 5/12 \rfloor = 28$, but `68.div(12).mul(5)` computes $\lfloor 68/12 \rfloor \times 5 = 5 \times 5 = 25$. IntentChecker evaluates the annotation to $[25, 25] \geq [28, 28]$ and reports Violated with risk score 10.0.

4.2.2 Exchange Rate Validation (Nokon)

Listing 2 shows a token purchase function where operator precedence causes the buyer to receive fewer tokens than the exchange rate warrants.

Listing 2: Nokon: exchange rate calculation with operator order issue (simplified).

```
1 function buy() public payable {
2     // @GlobalVar msg.value = [50000000000500000, ...]
3     // @StateVar presell = true
4     require(presell, "presell is closed");
5     uint256 amountToBuy = msg.value / ethRateFix
6                         * calculateRate();
7     // @During amountToBuy * ethRateFix >= msg.value * 250000
8     balances[msg.sender] += amountToBuy;
9 }
```

The developer's intent is that `amountToBuy` should satisfy the exchange-rate invariant: the tokens received, times the rate denominator, should be at least the ETH paid times the rate numerator. Because `msg.value / ethRateFix` truncates before multiplication, the invariant is violated for non-round input amounts. The @During annotation captures this invariant directly, and IntentChecker reports Violated.

4.2.3 Minting Invariant with Interface Call (Web3Bugs #78)

Listing 3 shows a rebasing vault’s mint function. The bug is that minted proxy tokens are computed from the total vault balance (including the just-deposited amount), which inflates the shares beyond the deposited amount when the redeem rate deviates from 1.

Listing 3: Web3Bugs #78-H-02: vault mint with inflated proxy amount (simplified).

```
1 function mint(address to, uint256 amount)
2   public override returns (uint256) {
3     // @LocalVar amount = [500e18, 500e18]
4     // @StateVar _totalSupply = [1000e18, 1000e18]
5     // @StateVar _balances[to] = [0, 0]
6     // @IReturn IERC20(baseToken).balanceOf()
7     //   = [1500e18, 1500e18]
8     uint256 _redeemRate = redeemRate();
9     require(IERC20(baseToken).transferFrom(
10      msg.sender, address(this), amount));
11     uint256 baseBalance =
12       IERC20(baseToken).balanceOf(address(this));
13     uint256 proxy = (baseBalance * ONE) / _redeemRate;
14     // @Post _balances[to] <= amount
15     _mint(to, proxy);
16 }
```

The `@Post _balances[to] := amount` annotation expresses the invariant that no user should receive more shares than the tokens they deposited. The `@IReturn` debug annotation injects a concrete balance (1500e18) for the external `balanceOf` call, which the analyzer would otherwise evaluate as $\top$ (§3.2(c)). With these annotations, `IntentChecker` computes the proxy amount and finds that it exceeds `amount`, reporting `Violated`. This example demonstrates how `@IReturn` enables intent validation for functions that depend on external contract state.

These three examples—drawn from real bugs in the evaluation benchmark—illustrate the two temporal scopes of intent validation: mid-execution checks (`@During` with relational and argument constraints) and post-execution checks (`@Post` combined with `@IReturn` for external state injection). The complete intent model supporting these annotations is defined in the following section.

4.3 Intent Model

The intent model consists of two complementary components that together determine both what a developer can write and what the written annotation means as a validation judgment. Section 4.3.1 fixes the concrete grammar of intent annotations and the informal meaning of each clause form, grouped by the two annotation types: `@During`, which checks conditions at specific program points within a function’s execution, and `@Post`, which checks conditions after the function completes. Section 4.3.2 then formalises how the validation engine evaluates an annotation against the abstract states produced by the interpreter, giving a denotational definition of each clause form as a tri-valued outcome in $\{S, V, W\}$. Together, the syntax specifies the surface language available to developers, and the semantics specifies the mathematical object that every syntactic form denotes.

duringIntent	→	// @During duringClause (logicOp duringClause)*
postIntent	→	// @Post postClause (logicOp postClause)*
duringClause	→	intentValue (before relOp after)
		intentValue (assign relOp current)
		identifier .arg[<i>n</i>] relOp intentValue
		commonClause
postClause	→	intentValue (entry relOp exit)
		commonClause
commonClause	→	returnExpression relOp intentValue
		return[<i>n</i>] relOp intentValue
		intentValue relOp PercentOf(intentValue , <i>p</i>)
		intentValue relOp ceil(intentValue , <i>d</i>)
		intentValue relOp floor(intentValue , <i>d</i>)
		intentValue relOp intentValue
		intentValue = _i intentValue
intentValue	→	arithExpr
arithExpr	→	arithExpr (+ -) arithTerm arithTerm
arithTerm	→	arithTerm (* / %) arithFactor arithFactor
arithFactor	→	number [number , number] varRef (arithExpr)
varRef	→	identifier subAccess*
subAccess	→	. identifier [expr]
logicOp	→	&& —
relOp	→	i i i= i= == != in not in

Fig. 2: Grammar of intent annotations.

4.3.1 Syntax

Annotation Syntax Overview. Figure 2 presents the grammar of intent annotations. Intent annotations are embedded in single-line comments, allowing them to coexist with Solidity code without affecting compilation. Each annotation begins with a marker (@During or @Post) followed by one or more clauses. Multiple clauses can be combined using logical operators: && (conjunction) requires all clauses to hold, while — (disjunction) requires at least one clause to hold.

Intent values within clauses support arithmetic expressions using standard operators (+, -, *, /, %), variable references with member and index access (e.g., balances[msg.sender]), numeric literals, and inline intervals written as [a, b] to represent value ranges directly. Relational operators include i, i, i=, i=, ==, !=, as well as in and not in for interval membership tests.

@During Annotation. The @During annotation specifies intended relationships at specific program points within a function’s execution. It supports the following clause types:

- $x(\text{before relOp after})$ —compares the value of variable x immediately before and after an assignment statement. This validates that assignments modify variables as intended (e.g., $\text{balance}(\text{before } i \text{ after})$ asserts that the balance decreases).
- $x(\text{assign relOp current})$ —compares the value being assigned to x against its current value. This is useful for validating incremental updates (e.g., $\text{counter}(\text{assign} == \text{current} + 1)$).
- $\text{func.arg}[n] \text{ relOp value}$ —validates the n -th argument passed to a function call, ensuring that function calls receive arguments satisfying specified constraints before execution proceeds.

@Post Annotation. The @Post annotation specifies intended relationships that should hold after function execution completes. It supports the following clause types:

- $x(\text{entry relOp exit})$ —compares the value of variable x at function entry against its value at function exit. This validates function-level effects on state variables (e.g., $\text{totalSupply}(\text{entry} == \text{exit})$ asserts that total supply remains unchanged).

Common Clauses. Both @During and @Post annotations share the following clause types:

- $\text{returnExpression relOp value}$ —validates the expression being returned against a specified value or range.
- $\text{return}[n] \text{ relOp value}$ —for functions returning multiple values (tuples), validates the n -th return value.
- $x \text{ relOp PercentOf}(y, p)$ —validates that x equals p percent of y , accounting for integer arithmetic. This is particularly useful for fee calculations and token distributions.
- $x \text{ relOp ceil}(y, d)$ and $x \text{ relOp floor}(y, d)$ —validate rounding behavior, where d specifies the divisor. These clauses help developers express intended rounding directions in division operations.
- $x \text{ relOp } y$ —general relational comparison between two intent values.
- $x =_i y$ —implication clause, asserting that if condition x holds, then condition y must also hold. This enables conditional intent specifications.

4.3.2 Semantics

We specify the meaning of intent annotations as a denotational function $\mathcal{V}[\![\cdot]\!]$ that maps each clause to a tri-valued validation outcome in $\{S, V, W\}$ (satisfied, violated, warning) under a validation context Γ . Throughout this subsection we use the point-wise convention $\mathcal{V}[\![\varphi]\!](\Gamma) = s$ to specify, for each syntactic form of φ , the state s produced by applying the meaning function to context Γ .

Abstract states and validation contexts. Let Σ denote the space of abstract variable environments produced by the interval-domain abstract interpreter: each $\sigma \in \Sigma$ maps every program variable to an interval (or to a structured value whose leaves are intervals). Let $\mathcal{I}$ denote the set of integer intervals. A validation context Γ is a record whose fields are listed in Table 2; each clause kind uses only the subset of fields relevant to its form.

Table 2: Fields of the validation context Γ . Each atomic clause kind references only a subset of these fields.

Field	Meaning
$\sigma_{\text{pt}} \in \Sigma$	abstract state at the annotated program point
$\sigma_{\text{before}} \in \Sigma$	state captured immediately before the statement at the annotated line
$\sigma_{\text{assign}} \in \Sigma$	initial-assignment environment for the enclosing function
$\sigma_{\text{entry}} \in \Sigma$	state at function entry (after parameter binding and debug annotations)
$\sigma_{\text{exit}} \in \Sigma$	joined state at normal function exit
$R : \text{Line} \rightarrow \text{Val}^*$	partial map from return-statement lines to captured return values
$\mathcal{C}$	the set of statements at the annotated line (for call-expression lookup)

For a `@During` annotation, Γ supplies σ_{pt} , σ_{before} , σ_{assign} , σ_{entry} , R , and $\mathcal{C}$; for a `@Post` annotation, Γ supplies σ_{entry} , σ_{exit} , and R . For clauses that may appear in either annotation type (the common clauses from Fig. 2), we let $\text{ref}(\Gamma)$ denote the reference environment: $\text{ref}(\Gamma) = \sigma_{\text{pt}}$ under `@During` and $\text{ref}(\Gamma) = \sigma_{\text{exit}}$ under `@Post`. This captures the rule that a common clause is evaluated at the annotated program point inside `@During` but at the function exit inside `@Post`.

Well-formedness. We write $\Gamma \vdash \varphi : \mathbf{wf}$ to mean that clause φ is well-formed under Γ : φ parses against Fig. 2, every variable referenced by φ resolves in the relevant environment of Γ , and all clause-specific auxiliary data required by Γ is present (a captured σ_{before} for `beforeAfter`, a return-map entry for `retExpr`, a matching call expression in $\mathcal{C}$ for `func.arg[n]`, and so on). The equations below define $\mathcal{V}[\![\varphi]\!](\Gamma)$ only for well-formed inputs; clause forms whose auxiliary data is missing correspond to error paths in the implementation and lie outside the semantic claim.

Well-formedness is orthogonal to the informativeness of the input intervals themselves. Debug annotations (`@StateVar`, `@LocalVar`, `@IReturn`), inherited from `SolQDebug`, are the mechanism by which developers supply bounded ranges for otherwise-unknown variables; without them, interval inputs may remain $\top$, and the comparison primitive defined below typically returns W even though the semantics is still sound. Debug annotations therefore raise the informativeness of validation results but are not a prerequisite for the semantics to be defined.

Expression evaluation. Intent-value expressions (Fig. 2, non-terminal `intentValue`) include ordinary arithmetic subexpressions, variable references with member and index access, interval literals $[a, b]$, and the DSL helpers `PercentOf`, `ceil`, and `floor`. They are evaluated against a chosen environment σ by a function

$$\text{eval} : \Sigma \times \text{intentValue} \rightarrow \mathcal{I},$$

which delegates to the interpreter’s expression evaluator for ordinary subexpressions (inheriting soundness of interval arithmetic from the base analyzer) and to annotation-specific rules for DSL-only constructs.

Comparison primitive. The core building block of the semantics is the set-theoretic comparison primitive

$$\text{Cmp}^* : \mathcal{I} \times \text{Op} \times \mathcal{I} \rightarrow \{S, V, W\},$$

defined as

$$\text{Cmp}^*(L, op, R) = \begin{cases} S & \text{if } \forall l \in L, \forall r \in R : l \text{ op } r, \\ V & \text{if } \forall l \in L, \forall r \in R : \neg(l \text{ op } r), \\ W & \text{otherwise.} \end{cases}$$

For $op \in \{\text{in}, \text{not in}\}$, the set-membership interpretation replaces the pointwise comparison: $l \text{ in } r$ holds iff $l \in R$, regardless of the particular r , and not in is its complement. By Theorems 3, 5, 6, and Corollary 7, Algorithm 4 implements Cmp^* exactly via the closed-form expressions of Table 4. The semantics thus has a sound computable realisation; conversely, Cmp^* gives an algorithm-independent specification of what the validation engine is required to compute.

Three-valued logic operators. Compound clauses combine sub-clauses via the connectives

$$\wedge_K, \vee_K, \supset_K : \{S, V, W\}^2 \rightarrow \{S, V, W\},$$

which coincide with Kleene’s strong three-valued logic and match the combination tables of Table 5. Treating W as the third truth value “unknown,” $\wedge_K$ yields S only when both arguments are S , yields V whenever any argument is V , and yields W otherwise; $\vee_K$ and $\supset_K$ are defined dually.

Meaning function. The meaning function

$$\mathcal{V}[\![\cdot]\!] : \text{Clause} \rightarrow (\text{Context} \rightarrow \{S, V, W\})$$

is defined by cases on the syntactic form of φ , one equation per clause kind.

@During clauses.

$$\begin{aligned} \mathcal{V}[x(\text{before } op \text{ after})](\Gamma) &= \text{Cmp}^*(\text{eval}(\sigma_{\text{before}}, x), op, \text{eval}(\sigma_{\text{pt}}, x)), \\ \mathcal{V}[x(\text{assign } op \text{ current})](\Gamma) &= \text{Cmp}^*(\text{eval}(\sigma_{\text{assign}}, x), op, \text{eval}(\sigma_{\text{pt}}, x)), \\ \mathcal{V}[f.\text{arg}[n] \text{ op } v](\Gamma) &= \text{Cmp}^*(\text{eval}(\sigma_{\text{pt}}, \text{arg}_n(f, \mathcal{C})), op, \text{eval}(\sigma_{\text{pt}}, v)), \end{aligned}$$

where $\text{arg}_n(f, \mathcal{C})$ denotes the n -th argument expression of the call to f located within the statements $\mathcal{C}$ at the annotated line.

@Post clauses.

$$\mathcal{V}[x(\text{entry } op \text{ exit})](\Gamma) = \text{Cmp}^*(\text{eval}(\sigma_{\text{entry}}, x), op, \text{eval}(\sigma_{\text{exit}}, x)).$$

Common clauses. The following equations apply uniformly to clauses appearing in either @During or @Post annotations; the reference environment $\text{ref}(\Gamma)$ selects σ_{pt} or

σ_{exit} as appropriate for the enclosing annotation type.

$$\begin{aligned}\mathcal{V}[\text{returnExpression } op \ v](\Gamma) &= \text{Cmp}^*(R(\ell_{\text{ret}}), \text{op}, \text{eval}(\text{ref}(\Gamma), v)), \\ \mathcal{V}[\text{return}[n] \text{ op } v](\Gamma) &= \text{Cmp}^*(R(\ell_{\text{ret}})[n], \text{op}, \text{eval}(\text{ref}(\Gamma), v)), \\ \mathcal{V}[e_l \text{ op } e_r](\Gamma) &= \text{Cmp}^*(\text{eval}(\text{ref}(\Gamma), e_l), \text{op}, \text{eval}(\text{ref}(\Gamma), e_r)),\end{aligned}$$

where ℓ_{ret} is the return-statement line selected by the annotation: in @During, the statement on the annotated line itself; in @Post, the join of all return values recorded in R .

Compound clauses.

$$\begin{aligned}\mathcal{V}[\varphi_1 \ \&\& \ \varphi_2](\Gamma) &= \mathcal{V}[\varphi_1](\Gamma) \ \wedge_K \ \mathcal{V}[\varphi_2](\Gamma), \\ \mathcal{V}[\varphi_1 \ \text{---} \ \varphi_2](\Gamma) &= \mathcal{V}[\varphi_1](\Gamma) \ \vee_K \ \mathcal{V}[\varphi_2](\Gamma), \\ \mathcal{V}[\varphi_1 \Rightarrow \varphi_2](\Gamma) &= \mathcal{V}[\varphi_1](\Gamma) \ \supset_K \ \mathcal{V}[\varphi_2](\Gamma).\end{aligned}$$

Connection to the algorithm. The validation engine of Section 4.4 computes $\mathcal{V}[\varphi](\Gamma)$ by traversing the clause structure: for each atomic clause it invokes Algorithm 4 with intervals selected according to the equations above; for each compound clause it applies the Kleene combination tables. By Corollary 7, the state returned by Algorithm 4 is exactly the state assigned by Cmp^* , so the engine’s output agrees with the semantic definition on every well-formed input. Algorithm 4 additionally computes the true probability p_{true} , which Algorithm 3 consumes to rank warnings for presentation; risk scoring is therefore a downstream concern that does not affect the semantic claim.

4.4 Validation Engine

The Validation Engine validates developer-specified intent annotations against actual program behavior computed through abstract interpretation. Given the CFG-based interval analysis from Section 4, the engine evaluates each annotation clause and determines whether the intended relationship holds across all possible execution states.

To perform validation, the engine captures variable environments at specific program points:

- `before_env`: variable state immediately before a statement executes
- `after_env`: variable state immediately after a statement executes (referred to as current in assign clauses)
- `assign_env`: the value being assigned in an assignment statement
- `entry_env`: variable state at function entry
- `exit_env`: variable state at function exit, computed by joining all normal termination paths

4.4.1 Validation Targets

@During Validation. The @During annotation is validated immediately after the associated statement executes:

- `var(before op after)`: compares `before_env` against `after_env` for variable `var`.
- `var(assign op current)`: compares `assign_env` against the current value of `var`.
- `func.arg[n] op value`: validates the n -th argument passed to the function call.

@Post Validation. The @Post annotation is validated when function execution completes:

- `var(entry op exit)`: compares `entry_env` against `exit_env` for variable `var`.

Common Clauses. The following clauses can be used in both @During and @Post annotations:

- `returnExpression op value`: compares the return value against the evaluated value.
- `return[n] op value`: compares the n -th element of a tuple return value against value.
- `exp1 op PercentOf(exp2, p)`: compares `exp1` against $p\%$ of `exp2`.
- `exp1 op ceil(exp2, d)` and `exp1 op floor(exp2, d)`: compares `exp1` against `exp2` rounded up or down to the nearest multiple of `d`.
- `exp1 op exp2`: general comparison between two expressions.

4.4.2 State Determination and Risk Scoring

The engine compares interval-domain values using Algorithm 4, which returns a pair $(state, p_{true})$ where *state* is one of three possible outcomes and $p_{true} \in [0, 1]$ is the true probability of the comparison holding under a uniform distribution assumption:

- *satisfied*: the condition holds for all values in the intervals ($p_{true} = 1$).
- *violated*: the condition fails for all values in the intervals ($p_{true} = 0$).
- *warning*: the condition holds for some values but not others ($0 < p_{true} < 1$).

The state alone determines the semantic validation result (Section 4.3.2); p_{true} serves as a downstream quantitative signal used only to rank warnings for presentation. Algorithm 3 transforms $(state, p_{true})$ together with structural information about the violated region into a risk score on a 0–10 scale, where 0.0 indicates a fully satisfied condition and 10.0 indicates a definite violation. For warning results, the engine classifies the violation pattern into three risk types based on the structure of the violated region: type 1 (single-direction, minor) maps to scores 0.1–3.3; type 2 (single-direction violation) maps to 3.4–6.6; and type 3 (bidirectional violation or equality operators) maps to 6.7–9.9. Within each band, the score is computed as $base + (1 - p_{true}) \times span$, so that lower p_{true} yields a higher score within the band.

Clause-to-Interval Mapping. Table 3 shows how each clause type maps to the interval comparison inputs L and R in Algorithm 4.

Interval Comparison Algorithm. Algorithm 4 presents the interval comparison procedure. Given two integer intervals $L = [l_{min}, l_{max}]$ and $R = [r_{min}, r_{max}]$ with an

Algorithm 3 Risk Scoring

```
1: Input: ( $state, p_{true}$ ) from Algorithm 4, operator  $op$ , false-region summary  $regions$ 
2: Output:  $risk\_score \in [0.0, 10.0]$ 

3: if  $state = satisfied$  then
4:   return 0.0
5: else if  $state = violated$  then
6:   return 10.0
7: end if

  // Warning case: classify and scale within band
8:  $risk\_type \leftarrow ClassifyRisk(regions, op)$ 
9: ( $base, span$ )  $\leftarrow BandOf(risk\_type)$   $\triangleright$  type 1: (0.1, 3.2); type 2: (3.4, 3.2); type 3: (6.7, 3.2)
10:  $risk\_score \leftarrow base + (1 - p_{true}) \times span$ 
11: return  $risk\_score$ 
```

Table 3: Mapping of clause types to interval comparison inputs. Here, exp_1 and exp_2 denote evaluated intent expressions; return value is the value returned by the current return statement; return value[n] is the n -th element when the function returns a tuple.

Clause Type	L	R
var(before op after)	before_env[var]	after_env[var]
var(assign op current)	assign_env[var]	after_env[var]
func.arg[n] op value	arg[n]	value
var(entry op exit)	entry_env[var]	exit_env[var]
returnExpression op value	return value	value
return[n] op value	return value[n]	value
exp_1 op exp_2	exp_1	exp_2

operator op , the algorithm partitions L into three regions under a uniform distribution assumption over both intervals:

- T_s (satisfied region): values of L for which $l \text{ op } r$ holds for all $r \in R$.
- T_v (violated region): values of L for which $l \text{ op } r$ fails for all $r \in R$.
- T_w (warning region): values of L whose truth value depends on the actual value of R .

Table 4 defines the early-return conditions and region sizes for each operator, where $n_L = l_{\max} - l_{\min} + 1$ and $n_R = r_{\max} - r_{\min} + 1$ denote the number of integer values in each interval.

Table 4: Per-operator satisfied/violated conditions and region sizes. All size expressions are clamped to $\max(0, \cdot)$. For in/not in, $|L \cap R|$ denotes the count of L -values within the range of R .

op	Satisfied condition	Violated condition	$ T_s $	$ T_v $
$>$	$l_{\min} > r_{\max}$	$l_{\max} \leq r_{\min}$	$l_{\max} - r_{\max}$	$r_{\min} - l_{\min} + 1$
$\geq$	$l_{\min} \geq r_{\max}$	$l_{\max} < r_{\min}$	$l_{\max} - r_{\max} + 1$	$r_{\min} - l_{\min}$
$<$	$l_{\max} < r_{\min}$	$l_{\min} \geq r_{\max}$	$r_{\min} - l_{\min}$	$l_{\max} - r_{\max} + 1$
$\leq$	$l_{\max} \leq r_{\min}$	$l_{\min} > r_{\max}$	$r_{\min} - l_{\min} + 1$	$l_{\max} - r_{\max}$
$=$	disjoint $\wedge$ singletons	disjoint	$ L \cap R $	$n_L - L \cap R $
$\neq$	disjoint	disjoint $\wedge$ singletons	$n_L - L \cap R $	$ L \cap R $
in	$L \subseteq R$	$L \cap R = \emptyset$	$ L \cap R $	$n_L - L \cap R $
not in	$L \cap R = \emptyset$	$L \subseteq R$	$n_L - L \cap R $	$ L \cap R $

For the comparison operators ($>$, $\geq$, $<$, $\leq$), the warning region T_w contains values of L whose comparison result depends on the actual value of R . Since Solidity operates on integers, for each $l \in T_w$ the number of R -values satisfying $l \text{ op } r$ forms an arithmetic sequence across the warning region. The expected true count E_w is therefore computed in closed form as $E_w = (a + b) \cdot |T_w| / (2 \cdot n_R)$, where a and b are the number of satisfying R -values for the smallest and largest elements of T_w , respectively.

For equality and inequality operators ($=$, $\neq$), each overlapping L -value has a $1/n_R$ probability of matching any particular R -value, rather than being a definite match. The probability is therefore adjusted by the factor $1/n_R$.

Compound Clauses and Implication. Multiple clauses can be combined using logical operators. The engine evaluates each clause independently and combines the three-valued results according to Table 5.

4.4.3 Algorithm Verification

We establish the correctness of Algorithm 4 in two complementary ways: a mathematical soundness proof showing that the closed-form expressions in Table 4 compute p_{true} exactly, and an empirical cross-validation against brute-force ground truth across a comprehensive battery of test cases.

Definitions. Let $L = [l_{\min}, l_{\max}]$ and $R = [r_{\min}, r_{\max}]$ be nonempty integer intervals with $n_L = l_{\max} - l_{\min} + 1$ and $n_R = r_{\max} - r_{\min} + 1$. For an operator op , let

$$N_{sat}(L, R, op) = |\{(l, r) \in L \times R : l \text{ op } r\}|$$

denote the number of satisfying pairs. Under a uniform distribution over $L \times R$, the true probability of satisfaction is

$$p_{true}(L, R, op) = \frac{N_{sat}(L, R, op)}{n_L \cdot n_R}.$$

Algorithm 4 Interval Comparison

```
1: Input: Intervals  $L = [l_{\min}, l_{\max}]$ ,  $R = [r_{\min}, r_{\max}]$ , operator  $op$ 
2: Output:  $(state, p_{true})$  where  $state \in \{satisfied, violated, warning\}$ 
3:  $n_L \leftarrow l_{\max} - l_{\min} + 1$ ,  $n_R \leftarrow r_{\max} - r_{\min} + 1$ 

    // Step 1: Early return for definite cases (Table 4)
4: if SatisfiedCond( $op, L, R$ ) then
5:   return (satisfied, 1.0)
6: else if ViolatedCond( $op, L, R$ ) then
7:   return (violated, 0.0)
8: end if

    // Step 2: Region partitioning (Table 4)
9:  $|T_s|, |T_v| \leftarrow$  per-operator formulas from Table 4
10:  $|T_w| \leftarrow n_L - |T_s| - |T_v|$ 

    // Step 3: Compute  $p_{true}$ 
11: if  $op \in \{\text{in}, \text{not in}\}$  then
12:    $p_{true} \leftarrow |T_s| / n_L$ 
13: else if  $op \in \{=, \neq\}$  and  $n_R > 1$  then
14:    $p_{true} \leftarrow |T_s| / (n_L \cdot n_R)$   $\triangleright = \text{case}; \neq: 1 - |T_v| / (n_L \cdot n_R)$ 
15: else  $\triangleright >, \geq, <, \leq$ 
16:    $E_w \leftarrow$  expected true count from  $T_w$  via arithmetic series over  $R$ 
17:    $p_{true} \leftarrow (|T_s| + E_w) / n_L$ 
18: end if
19: return (warning,  $p_{true}$ )
```

Algorithm 4 is sound if its returned value p_{func} equals p_{true} for every well-formed input.

Region partition. For $op \in \{>, \geq, <, \leq\}$, the algorithm decomposes L into three disjoint regions instantiating the prose given earlier:

$$\begin{aligned} T_s &= \{l \in L : l \text{ op } r \text{ holds for all } r \in R\}, \\ T_v &= \{l \in L : l \text{ op } r \text{ fails for all } r \in R\}, \\ T_w &= L \setminus (T_s \cup T_v). \end{aligned}$$

The decomposition is exhaustive: for fixed $l \in L$, the truth value of $l \text{ op } r$ is monotonic in r , so it is either always true ($l \in T_s$), always false ($l \in T_v$), or transitions exactly once as r ranges over R ($l \in T_w$).

Lemma 1 (Region sizes for $>$) For $op = >$, in the warning case ($l_{\min} \leq r_{\max}$ and $l_{\max} > r_{\min}$),

$$|T_s| = \max(0, l_{\max} - r_{\max}), \quad |T_v| = \max(0, r_{\min} - l_{\min} + 1).$$

Table 5: Three-valued logic combination rules for compound clauses. S = satisfied, V = violated, W = warning.

$\&\&$	S	V	W
S	S	V	W
V	V	V	V
W	W	V	W
—	S	V	W
S	S	S	S
V	S	V	W
W	S	W	W
$=_l$	S	V	W
S	S	V	W
V	S	S	S
W	W	W	W

Proof $T_s = \{l \in L : l > r_{\max}\} = L \cap [r_{\max} + 1, \infty) = [\max(l_{\min}, r_{\max} + 1), l_{\max}]$, whose cardinality is $\max(0, l_{\max} - r_{\max})$. Similarly, $T_v = L \cap (-\infty, r_{\min}] = [l_{\min}, \min(l_{\max}, r_{\min})]$, with cardinality $\max(0, r_{\min} - l_{\min} + 1)$. $\square$

Lemma 2 (Sum over T_w for $>$) *Under the same assumptions, let $T_w = [l_w^-, l_w^+]$ and define $a = l_w^- - r_{\min}$, $b = l_w^+ - r_{\min}$. Then*

$$\sum_{l \in T_w} |\{r \in R : l > r\}| = \frac{(a+b)|T_w|}{2}.$$

Proof For $l \in T_w$ we have $r_{\min} < l \leq r_{\max}$, so $\{r \in R : r < l\} = [r_{\min}, l-1]$, with cardinality $l - r_{\min}$. Summing over $l \in T_w$ yields an arithmetic progression:

$$\sum_{l=l_w^-}^{l_w^+} (l - r_{\min}) = \sum_{k=a}^b k = \frac{(a+b)(b-a+1)}{2} = \frac{(a+b)|T_w|}{2}. \quad \square$$

Theorem 3 (Soundness for $>$) *For $op = >$, $p_{func} = p_{true}$.*

Proof Case 1 ($l_{\min} > r_{\max}$). Every pair satisfies $l > r$, so $N_{sat} = n_L \cdot n_R$ and $p_{true} = 1$. Algorithm 4 takes the satisfied early return and outputs $p_{func} = 1$.

Case 2 ($l_{\max} \leq r_{\min}$). No pair satisfies $l > r$, so $N_{sat} = 0$ and $p_{true} = 0$. The algorithm takes the violated early return and outputs $p_{func} = 0$.

Case 3 (warning case). Decompose

$$N_{sat} = \sum_{l \in L} |\{r \in R : l > r\}| = \underbrace{|T_s| \cdot n_R}_{l \in T_s} + \underbrace{\sum_{l \in T_w} (l - r_{\min})}_{l \in T_w} + \underbrace{0}_{l \in T_v},$$

where the first term uses the fact that every $l \in T_s$ satisfies $l > r$ for every $r \in R$. By Lemma 2, the middle sum equals $(a+b)|T_w|/2$. Dividing by $n_L \cdot n_R$ gives

$$p_{true} = \frac{|T_s|}{n_L} + \frac{(a+b)|T_w|}{2n_L n_R} = \frac{|T_s| + E_w}{n_L},$$

where $E_w = (a+b)|T_w|/(2n_R)$ is exactly the quantity computed by Algorithm 4. Hence $p_{func} = p_{true}$. $\square$

Corollary 4 (Soundness for $\geq, <, \leq$) *The remaining comparison operators $op \in \{\geq, <, \leq\}$ admit symmetric proofs: the strict inequality is replaced by its non-strict counterpart, or the roles of L and R are swapped. The closed forms of $|T_s|$ and $|T_v|$ in Table 4 arise from Lemma 1 via these substitutions, and the contribution from T_w is computed by the same arithmetic-series argument as Lemma 2, yielding $p_{func} = p_{true}$ in each case.*

Theorem 5 (Soundness for $=$ and $\neq$) *For $op \in \{=, \neq\}$, $p_{func} = p_{true}$.*

Proof For $op =$, each value $v \in L \cap R$ contributes exactly one satisfying pair (v, v) , and no other pair satisfies $l = r$. Hence $N_{sat} = |L \cap R|$ and

$$p_{true}(=) = \frac{|L \cap R|}{n_L \cdot n_R}.$$

When $n_R > 1$, Algorithm 4 branches on $|T_s|/(n_L \cdot n_R)$, with $|T_s| = |L \cap R|$ from Table 4, matching p_{true} . The case $op \neq$ follows by complementation: $p_{true}(\neq) = 1 - p_{true}(=) = 1 - |L \cap R|/(n_L \cdot n_R)$, which is exactly the complementary expression noted in Table 4.

When $n_R = 1$, the algorithm bypasses the dedicated $n_R > 1$ branch and falls through to the general comparison path. The warning region T_w is then necessarily empty (since $|T_s| + |T_v| = |L \cap R| + (n_L - |L \cap R|) = n_L$), so $E_w = 0$ and the formula reduces to $|T_s|/n_L = |L \cap R|/n_L$. Direct calculation confirms this equals p_{true} when $n_R = 1$: each $l \in L$ contributes one satisfying pair iff $l = r_{\min}$, giving $N_{sat} = |L \cap R|$ and $p_{true} = |L \cap R|/n_L$. $\square$

Theorem 6 (Soundness for in and not in) *For $op \in \{\text{in}, \text{not in}\}$, $p_{func} = p_{true}$.*

Proof The semantics of in treats R as a set: $l \in R$ holds iff $r_{\min} \leq l \leq r_{\max}$, independent of any particular r . Each $l \in L \cap R$ therefore yields n_R satisfying pairs (one for every r), so $N_{sat} = |L \cap R| \cdot n_R$ and

$$p_{true}(\text{in}) = \frac{|L \cap R| \cdot n_R}{n_L \cdot n_R} = \frac{|L \cap R|}{n_L} = \frac{|T_s|}{n_L},$$

which matches the algorithm's branch for in . The case not in is the complement, $1 - p_{true}(\text{in})$. $\square$

Corollary 7 (State consistency) *Algorithm 4 returns (state, p_{true}) where state matches the set-theoretic definition of the comparison primitive Cmp^* given in Section 4.3.2: $\text{state} = \text{satisfied}$ iff every pair $(l, r) \in L \times R$ satisfies $l \text{ op } r$; $\text{state} = \text{violated}$ iff no pair does; and $\text{state} = \text{warning}$ otherwise.*

Proof By Theorems 3, 5, and 6, Algorithm 4 computes p_{true} exactly under the uniform distribution. The state branches are then determined by the boundary cases of p_{true} : $p_{true} = 1$ iff every pair satisfies the comparison (the set-theoretic satisfied condition), $p_{true} = 0$ iff no pair satisfies (the violated condition), and $0 < p_{true} < 1$ iff both satisfying and non-satisfying pairs exist (the warning condition). These three cases match the corresponding early-return and fall-through branches of the algorithm. $\square$

Compositional soundness. Compound clauses are built from atomic clauses via the connectives $\wedge$, $\vee$, and $\Rightarrow$, evaluated under the three-valued logic of Table 5. Treating warning as the third truth value “unknown,” these tables coincide with Kleene’s strong three-valued logic, a standard result whose soundness with respect to its underlying classical interpretation is well known: a compound clause evaluates to satisfied (resp. violated) only when its classical reading would be true (resp. false) under every disambiguation of warning. By induction on clause structure, if every atomic sub-clause is evaluated soundly by Algorithm 4, the compound clause is evaluated soundly as well.

Edge cases. Two boundary conditions deserve mention. (i) Empty intervals. An empty L or R corresponds to an infeasible execution path; the abstract interpreter prunes such branches before invoking Algorithm 4, so the algorithm is never called on empty inputs. (ii) Singleton right interval. When $n_R = 1$, the comparison operators $\{>, \geq, <, \leq\}$ reduce to standard pointwise tests handled by the early returns; the warning region T_w degenerates to at most one element, and Lemma 2 continues to hold (the arithmetic series is trivial). The equality operators $\{=, \neq\}$ with $n_R = 1$ are handled by Theorem 5. Together, Theorems 3, 5, and 6, Corollaries 4 and 7, and the compositional argument above establish that Algorithm 4 computes p_{true} exactly and returns a state matching Cmp^* for every operator and every well-formed input.

Empirical cross-validation. To guard against transcription errors and unforeseen edge cases in the formal development above, we additionally verified Algorithm 4 on a comprehensive battery of test cases that exhaustively cover the topological configurations of two integer intervals.

Interval topology. Given two integer intervals $L = [l_{\min}, l_{\max}]$ and $R = [r_{\min}, r_{\max}]$, their relative position falls into exactly one of eleven topological configurations, determined by how the endpoints relate. We organize them into three groups based on the number of shared endpoints (Figure 3): (a) no shared endpoints (six configurations covering separation, partial overlap in either direction, and strict containment in either direction), (b) one shared endpoint (four configurations covering left- or right-shared with L or R being the wider interval), and (c) both shared endpoints (the identical case).

Verification method. For each of the eleven topological configurations, we generate five width profiles that systematically vary the interval sizes: Small (width 1–3), Medium (5–10), Large (20–50), Asymmetric-L ($|L| \gg |R|$), and Asymmetric-R ($|R| \gg |L|$). Each configuration–profile pair is tested against all eight comparison operators ($>, \geq, <, \leq, =, \neq, \text{in}, \text{not in}$), yielding $11 \times 5 \times 8 = 440$ test cases. For every test case, we compute the ground-truth probability p_{bf} by exhaustively enumerating all (l, r) integer pairs under a uniform distribution and compare it against the value p_{func} returned by Algorithm 4. A test passes when $|p_{bf} - p_{func}| < 0.001$.

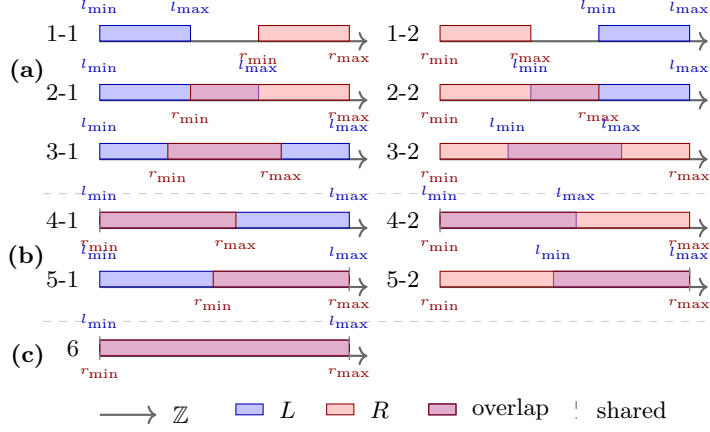


Fig. 3: Eleven topological configurations of two integer intervals, grouped by the number of shared endpoints: (a) no shared endpoints (six configurations), (b) one shared endpoint (four configurations), and (c) both shared endpoints (the identical case). Overlapping regions appear as blended color; dashed vertical lines indicate shared endpoints.

Table 6: Brute-force verification results, grouped by topological category. Each category aggregates its constituent configurations across 5 width profiles $\times$ 8 operators per configuration.

Topology category	Tests	Pass	Fail
(a) No shared endpoints (6 configurations)	240	240	0
(b) One shared endpoint (4 configurations)	160	160	0
(c) Both shared endpoints (1 configuration)	40	40	0
Total	440	440	0

Result. All 440 test cases pass with an exact match between the brute-force ground truth and the algorithm output (Table 6). The 100% pass rate across all configurations and width profiles cross-validates the soundness proof above, confirming that the closed-form expressions of Table 4 indeed compute p_{true} for every possible relative position of two integer intervals, independent of their absolute sizes.

5 Evaluation

The validation algorithm itself was already verified in §4.4.3, where we proved that the closed-form expressions of Table 4 compute p_{true} exactly and cross-validated this proof against brute-force ground truth across every topological configuration of two

Table 7: Benchmark dataset composition.

Source	Error Pattern	Count
NumScout	Div In Path	1
	Exchange Problem	1
	Operator Order Issue	1
	Precision Loss Trend	2
	Minor Amount Retention	1
	Profit Opportunity	1
Flyinointment	Greedy Contract	1
Web3Bugs	Erroneous Accounting	72
	Inconsistent State Updates	9
Total (collected)		89
Excluded (see below)		−14
Effective		75

Contract, so this single instance is sourced separately to ensure pattern-level coverage of all the single-transaction error patterns described in §2.1.

Web3Bugs (81 instances). Web3Bugs (Zhang et al., 2023) systematically collected and classified 493 high-severity vulnerabilities from Code4rena audit contests. We restrict the candidate set to bugs whose execution context fits IntentChecker’s scope, namely multi-contract single-transaction numeric logic errors. This corresponds to (i) all bugs in categories S6-1 through S6-4 (Erroneous Accounting, 86 bugs), whose root causes are semantic errors in arithmetic or accounting logic and which the Web3Bugs authors confirm to be single-transaction, and (ii) the subset of S3-1 (Inconsistent State Updates) bugs whose root cause involves a numeric computation with a missing or stale state variable update. We manually reviewed the 22 S3-1 bugs and retained the 11 instances meeting this criterion, excluding non-numeric cases such as approval resets and access-control omissions. This yields 97 candidate instances.

We then exclude 16 instances whose target contracts contain inline assembly blocks—either directly in the target contract or in locally imported/inherited files—which cannot be processed by the abstract interpreter (the supported Yul subset is described in §3.2). After these exclusions, the final Web3Bugs subset contains 81 instances spanning Erroneous Accounting (72) and Inconsistent State Updates (9).

Dataset Summary. Table 7 summarises the resulting benchmark. For each instance, we record the target contract, the buggy function, the specific lines identified in the audit report, and the error pattern.

Exclusions. Of the 89 collected instances, 14 are excluded after manual inspection because they fall outside the definition of a development-time numeric logic error. The exclusion categories are: source already silently patched (E1, 2 cases); checked-arithmetic revert under Solidity ≥ 0.8 such that the program traps rather than returns a wrong value (E2, 2 cases); duplicate audit findings on the same buggy site (E3, 2 cases); reports the original auditors disputed as intended behavior (E4, 1 case); required external libraries not provided in the audit package (E5, 2 cases); bugs whose manifestation requires state changes across separate transactions and therefore fall outside our single-transaction scope (E6, 3 cases); a case whose bug actually resides

in a different contract than the documented target (1 case); and one NumScout case whose “bug” is the inherent truncation of integer division for which no alternative correct implementation exists (E7, 1 case). The remaining **75 instances** form the effective evaluation set used in RQ1–RQ3.

Methodology. For each of the 75 effective instances, we (1) analyse the audit report to understand the root cause, (2) write an intent annotation that captures the expected numeric behaviour described in the report, and (3) run IntentChecker to record whether it reports Satisfied, Violated, or Warning. Cases that produce a non-Satisfied verdict are reported as mitigated in RQ1; cases for which no annotation can yield such a verdict are classified into the limitation taxonomy (§5.2) and analyzed for annotation expressibility in RQ2.

5.2 RQ1: Mitigation

Setup. For each of the 75 effective benchmark instances (§5.1), we (1) read the audit report and identify the buggy line(s), (2) write @During and/or @Post intent annotations that capture the expected numeric behavior described in the report, (3) supply the debug annotations (@StateVar, @LocalVar, @GlobalVar, and @IReturn) that the function needs to be analyzable under representative inputs, and (4) run IntentChecker on the resulting annotated source. We record the verdict produced by the validation engine—Satisfied, Warning, or Violated—together with the risk score and violated region. For instances where no annotation can yield a non-Satisfied signal, we further classify the structural reason according to the limitation taxonomy described below.

Outcomes. Of the 75 effective instances, IntentChecker produces a Violated verdict on **20 cases** when supplied with annotations derived from the audit report. The remaining 55 instances are classified as not_detectable: IntentChecker cannot, given the structure of its analysis or its intent model, surface a non-Satisfied signal for them. We first describe the 20 mitigated cases in detail (pattern coverage, annotation usage, and analysis time) and then present the limitation taxonomy for the 55 unmitigated cases.

Pattern coverage of the 20 mitigated cases. The 20 mitigated instances cover 7 of the 8 single-transaction error patterns enumerated in §2.1: Div In Path, Operator Order Issue, Exchange Problem, Precision Loss Trend, Minor Amount Retention, Erroneous Accounting (with both Wrong Interest Rate Order and Wrong Checkpoint Order sub-patterns appearing), Inconsistent State Updates, and Greedy Contract. Profit Opportunity is present in the benchmark but its representative case falls into the L4 category and is not mitigated. This pattern-level distribution confirms that the mitigated cases are not concentrated in a single “easy” pattern but span the breadth of the taxonomy, suggesting that intent-based validation is a general technique rather than a coincidental match for one error class.

Annotation model usage on the 20 mitigated cases. Table 8 reports which intent clauses were used to surface the bug in each mitigated case. Of the 20 cases, 12 use a @During clause (mid-execution checks) and 8 use a @Post clause (post-execution checks). The single most common form is the simple relational comparison (intent-Value relOp intentValue), used in 9 cases across both @During and @Post. The two

Table 8: Intent clause forms used in the 20 mitigated cases. Counts are per case (one annotation per case unless otherwise noted; one case uses two annotations of the same form on adjacent statements).

Annotation type	Clause form	Count
@During (12 cases)		
	Relational comparison (var relOp expr)	5
	Before/After (var(Before relOp After))	2
	Function-arg check (func.arg[N] relOp expr)	3
	Implication form	1
	Feasibility check (require feasible)	1
@Post (8 cases)		
	Relational comparison (var relOp expr)	4
	Entry/Exit (var(Entry relOp Exit))	2
	Return expression (returnExpression relOp expr)	1
	changed(var, true/false)	1
Total		20

specialized @During forms—Before/After comparisons against an injection point, and func.arg[N] checks on argument values—are each used 2–3 times, and the specialized @Post forms (Entry/Exit, returnExpression, changed()) account for the remainder. This distribution suggests that the bulk of mitigation work in our benchmark is carried by the basic relational form, with the specialized clauses contributing on patterns that the basic form cannot capture (e.g., monotonicity over function execution, argument-level constraints, post-execution change detection).

Analysis time and what drives it. We measured per-case end-to-end analysis time over three independent runs of each mitigated case on a single thread, excluding the one-time dependency pre-analysis phase (consistent with the development-time scenario, in which inherited contracts and libraries are cached across editing sessions). The mean per-case time is **3.69 s**, the minimum is 0.33 s (web3bugs_77_H.01), and the maximum is 14.94 s (div_in_path_WANGMI); 15 of the 20 cases finish in under 5 s, and all but two finish in under 10 s. Per-case standard deviations are small for short cases (under 0.15 s) and larger for the long-running cases (e.g., web3bugs_45_H.01 at 13.5 ± 4.5 s), reflecting OS and JIT noise that becomes proportionally smaller as case complexity grows. The full per-case timings appear in Table 9.

To understand what drives the variation across cases, we collected six per-case complexity metrics—total source lines in the deployment unit, function count, control-flow branch count, loop count, debug annotation count, and external call count—and computed the Pearson correlation of each with the mean analysis time. Figure 4 plots each metric against analysis time. Three observations emerge. First, the strongest predictors are aggregate size measures: source lines ($r = 0.87$), function count ($r = 0.76$), and branch count ($r = 0.72$). The dominant cost is therefore the abstract interpreter’s traversal of the deployment unit’s control-flow graph, not any single feature local to the bug site. Second, debug annotation count is also strongly correlated ($r = 0.80$),

Table 9: Per-case analysis time on the 20 mitigated cases (mean $\pm$ stdev over 3 runs, single thread, dependency pre-analysis excluded). Cases sorted by mean time.

Case	Mean (s)	Stdev (s)	Range (s)
div_in_path_WANGMI	14.94	2.56	12.0–16.8
web3bugs_45_H_01	13.55	4.52	9.4–18.4
web3bugs_58_H_02	9.67	3.66	7.0–13.8
indivisible_amount_BoostToken	5.67	0.60	5.0–6.1
operator_order_issue_BoostToken	5.43	0.48	4.9–5.8
web3bugs_62_H_08	4.76	0.32	4.4–5.1
web3bugs_70_H_10	3.32	0.31	3.0–3.6
web3bugs_51_H_02	3.18	0.69	2.8–4.0
web3bugs_56_H_02	2.37	1.45	1.5–4.0
profit_opportunity_HIT	2.03	0.46	1.6–2.5
web3bugs_5_H_12	1.75	1.41	0.9–3.4
web3bugs_47_H_02	1.42	0.11	1.3–1.5
web3bugs_5_H_08	1.10	0.17	1.0–1.3
web3bugs_60_H_01	1.06	0.07	1.0–1.1
web3bugs_101_H_01	0.87	0.04	0.8–0.9
exchange_problem_Nokon	0.67	0.09	0.6–0.8
greedy_contract_SwordCrowdsale	0.66	0.07	0.6–0.7
web3bugs_78_H_02	0.58	0.02	0.6–0.6
web3bugs_5_H_07	0.44	0.12	0.3–0.6
web3bugs_77_H_01	0.33	0.02	0.3–0.4
Mean of means	3.69	—	—

reflecting that each debug annotation injects an additional state-evaluation point into the interpreter and thus contributes proportionally to total work. Third, and perhaps counter-intuitively, loop count is only moderately correlated ($r = 0.36$), and external call count only weakly so ($r = 0.38$). This contradicts the natural expectation that fixpoint iteration on loops would dominate analysis time. Inspection of the mitigated set explains the gap: in several cases that contain loops, the widening of accumulator variables happens to occur on variables that do not flow into the intent annotation’s value path, so the precision loss does not affect the verdict but the fixpoint iteration cost is still paid. The cost is real, but it is not concentrated in the cases that succeed—those cases instead pay it as “dead work” alongside successful bug surfacing. A future optimization that confines fixpoint iteration to the intent-relevant scope could reduce this overhead without changing detectability; we discuss this further in §6.2.

Per-tool latency comparison against NumScout and GPTScan is reported in §5.4.

Limitation taxonomy for the 55 unmitigated cases. The structural reasons for the 55 not_detectable instances fall into eleven sub-categories grouped into two top-level families, summarized in Figure 5.

Limitation type definitions.

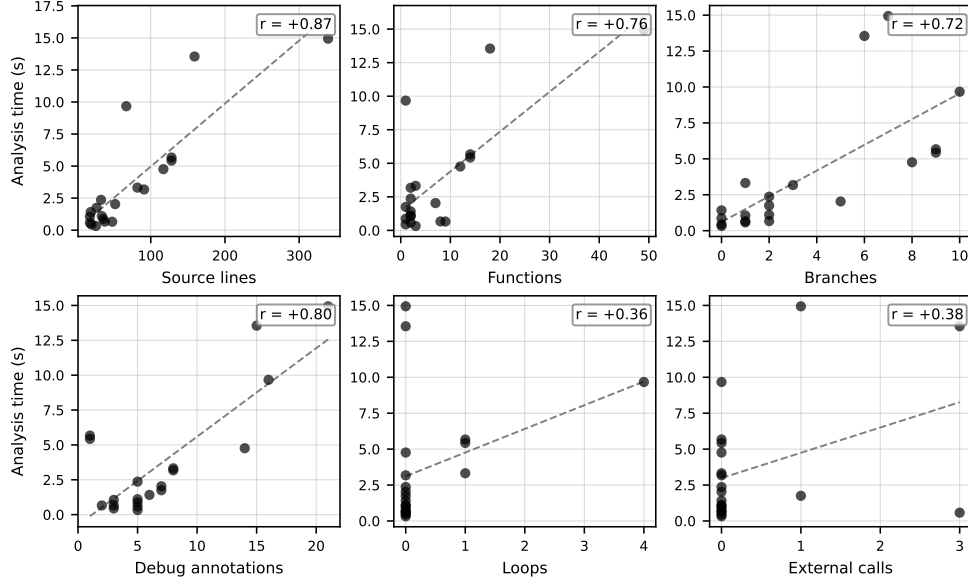


Fig. 4: Scatter plots of per-case analysis time against six complexity metrics (20 mitigated cases, 3-run mean). Dashed lines show linear regression; r is the Pearson correlation coefficient.

- **L1a (Loop widening, 8).** Accumulator variables ($+=$, $*=$) inside fixpoint iteration are widened to $\top$, and both buggy and correct executions land in the same widened range.
- **L1b (Loop body granularity, 2).** The bug is exposed only at a specific point inside a loop body, but the analyzer summarizes loops by fixpoint result rather than by per-iteration trace, so no annotation point is available at the granularity needed.
- **L2a (Interface call return $\top$, 3).** The callee is reached only through an interface variable, the implementation is therefore unavailable, and `@IReturn` is inapplicable because the bug requires the return value of a state-modifying call.
- **L2b (External call, unknown external state, 1).** The implementation exists via dependency pre-analysis, but the callee’s runtime storage state lies outside the deployment unit and the return value collapses to $\top$.
- **L3 (Unsupported construct, 7).** The bug-relevant value flows through an unsupported construct—`abi.decode`, opaque hash builtins, `address(this).balance`, or non-arithmetic Yul—which the analyzer treats as $\top$.
- **L4a (Inexpressible expected value, 10).** The intent annotation grammar can refer to in-scope arithmetic over existing variables, but the correct expected value depends on a new intermediate computation, an external contract’s state, or a function call that does not appear in the program; no `intentValue` expression can be constructed.
- **L4b (No target storage variable, 8).** The buggy function (a view function, a pure function, a thin wrapper around external calls, or a library helper) does not

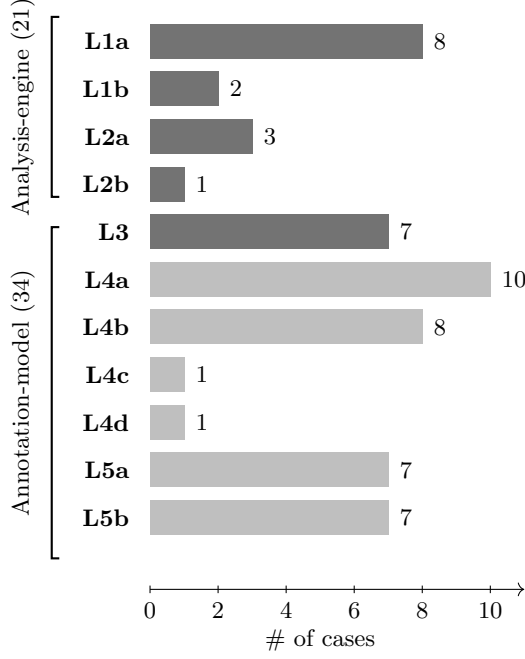


Fig. 5: Limitation breakdown for the 55 not_detectable instances. Analysis-engine limitations (L1–L3, 21 cases) are precision losses internal to the abstract interpreter that no developer annotation can recover. Annotation-model limitations (L4–L5, 34 cases) are cases the interpreter would distinguish given the right annotation, but the current model cannot express that annotation (L4) or only with bug awareness (L5).

modify any storage variable in the target contract, so no @Post clause has anything to attach to.

- **L4c (Magnitude-only difference, 1).** Buggy and correct code change the same state variable in the same direction; only the magnitude differs. `changed()` and `Entry op Exit` both succeed in both versions, and `IntentChecker`’s `PostEntryExit` clause does not currently admit arithmetic relations between `Entry` and `Exit`.
- **L4d (Invariant masked, 1).** A different statement in the same function already modifies the target state variable, so `changed()` is satisfied in both versions. The actual invariant is a multi-variable arithmetic relation (e.g., a product invariant) which `PostEntryExit` cannot express.
- **L5a (Bug-awareness-required: missing code, 7).** A required statement (a function call, a state update) is omitted; the post-condition that would catch this is expressible, but knowing what is missing already presupposes knowing the bug.
- **L5b (Bug-awareness-required: wrong code, 7).** A required statement is present but uses the wrong identifier, operator, or struct field; the corrective annotation is expressible, but writing it requires knowledge of the correct form—and possessing that knowledge would have prevented the bug in the first place.

Reading the limitation breakdown. Two top-level families emerge. The L1–L3 limitations (21 instances combined) are properties of the underlying abstract interpreter: the analyzer assigns $\top$ to a value that should have been concrete enough to distinguish buggy from correct execution, and no developer annotation under the current model can recover that precision. The L4–L5 limitations (34 instances) are properties of the intent model rather than of the analyzer: the abstract interpreter would distinguish the buggy and correct executions if the developer could express the right intent, but the current grammar either has no expression the developer could write (L4) or has one whose construction presupposes already knowing the bug (L5). RQ2 examines this second family in detail, classifying each of the 34 cases by whether the intent model can express any awareness signal at all—and if so, how directly.

5.3 RQ2: Annotation Model Expressibility

Setup. RQ1 partitioned the 55 not_detectable cases into two top-level families: 21 cases that fail because the abstract interpreter loses precision (L1–L3, analysis-engine limitations) and 34 cases that fail because the intent model cannot express the right intent (L4–L5). RQ2 examines the second family in detail. The question is no longer “did mitigation succeed?”—it did not, by construction—but rather: if a developer were already aware of the bug, could they at least express an annotation that the validation engine would surface as Violated or Warning? This separates the model’s expressibility ceiling from practical detectability, which depends on whether a developer would write that annotation in the first place.

Three-tier classification. For each of the 34 L4–L5 cases we attempted to construct an annotation, assuming full bug awareness, that would yield a non-Satisfied verdict on the buggy code. Each case is then placed into one of three tiers based on the strength of the resulting signal.

- **Inexpressible (L4).** No annotation under the current grammar can yield a non-Satisfied verdict, even with full bug awareness. Either the correct value cannot be constructed from in-scope variables (L4a), the buggy function modifies no target storage (L4b), the difference is in magnitude only along the same direction (L4c), or another statement in the same function masks the change (L4d).
- **Expressible-Strong (L5, “Direct”).** The annotation expresses the correct value or relation directly (a commonClause of the form `intentValue relOp intentValue`). The buggy version produces a clean Violated; the corrected version produces Satisfied. The annotation is essentially a fix-equivalent specification.
- **Expressible-Weak (L5, “Indirect”).** The annotation captures only that some state variable changed—or did not change—in a particular direction (`changed(var, true/false)` or `var(Entry relOp Exit)`). The buggy version produces Violated because the change does not occur in the expected way, but the annotation does not encode the correct value, only its presence or polarity. We call this an awareness signal: it is enough to alert the developer that something is wrong, even though it does not tell them what the correct value is.

Results. The three-tier classification of the 34 L4–L5 cases is summarized in Table 10. Inexpressible cases account for 20 of 34 (59%); Strong and Weak each account for 7

Table 10: RQ2 expressibility classification of the 34 intent-model limitation cases (L4–L5). Inexpressible: no annotation yields a non-Satisfied verdict even with full bug awareness. Strong: the annotation encodes the correct value directly. Weak: the annotation provides an awareness signal (change detection) without encoding the correct value.

Tier / Sub-category	Count	%
Inexpressible (L4)	20	59
L4a: inexpressible expected value	10	
L4b: no target storage variable	8	
L4c: magnitude-only difference	1	
L4d: invariant masked	1	
Expressible-Strong (Direct)	7	21
L5a (missing code)	2	
L5b (wrong code)	5	
Expressible-Weak (Indirect)	7	21
L5a (missing code)	5	
L5b (wrong code)	2	
Total	34	100

(21% each). Within the L5 group alone, all 14 cases are expressible—14 of 14, evenly split between Strong (7) and Weak (7)—which means the intent model can encode some awareness signal for every case where bug awareness is the only obstacle. The Inexpressible tier maps exactly to the L4 sub-categories (L4a 10, L4b 8, L4c 1, L4d 1), confirming that L4 represents a hard expressibility ceiling rather than a question of developer awareness.

Representative examples. We illustrate one case from each tier to show the qualitative difference in signal.

Inexpressible (L4a) — web3bugs_51_H_04 (SwapUtils.getYC). The bug is that when a swap crosses a target price, the StableSwap formula switches its amplifier coefficient mid-trade; the correct behavior is to split the swap at the crossing point and apply each amplifier to its sub-swap. Constructing this requires computing a split point dx_1 that does not appear anywhere in the program and that is not expressible as a closed-form arithmetic combination of in-scope variables (it is the solution to a Newton-method equation). No intentValue expression can capture the correct getY return, so the annotation grammar’s intentValue == arithExpr form is unusable here. Expressible-Strong (L5b) — web3bugs_113_H_05 (NFTPairWithOracle.lend). The buggy line is require(params.ltvBPS != accepted.ltvBPS, ...); the correct comparison is j= (a lender wants the borrower’s LTV to be at most their accepted ceiling). Under bug awareness, the annotation is direct:

```
// @During params.ltvBPS <= accepted.ltvBPS
```

The buggy code admits inputs with `params.ltvBPS` i accepted.ltvBPS, which violate the annotation cleanly. The annotation is essentially the corrective require. Expressible-Weak (L5a) — `web3bugs_192_H.01` (`Lock.extendLock`). The function performs `transferFrom` for an additional `_amount` but forgets to update the internal accounting variable `totalLocked[_asset]`. Under bug awareness, the developer can write

```
// @Post totalLocked[_asset](Entry < Exit)
```

which fires `Violated` on the buggy version (the variable does not change, so `Entry == Exit`) and is satisfied on a corrected version that adds `totalLocked[_asset] += _amount`. The annotation tells the developer that the variable was supposed to grow, but it does not tell them by how much—the precise increment `_amount` is not present in the annotation. This is an awareness signal, not a fix-equivalent specification, and our grammar does not currently support `intentValue` expressions over the `Entry` value, which would be needed to encode the increment exactly.

Implications. Two findings stand out. First, the distinction between L4 and L5 is meaningful: L4’s 20 cases are a hard ceiling that no amount of developer awareness can lift, whereas L5’s 14 cases are a soft ceiling that the intent model already covers, contingent on whether the developer is aware. Second, within L5, the 50/50 split between Strong and Weak suggests that for half of the awareness-conditional cases, the developer’s annotation would express the actual fix, and for the other half it would express a coarser “something must change here” signal that nevertheless suffices to surface the bug. Future grammar extensions that would enlarge the Strong tier and shrink Inexpressible—specifically, supporting (a) arithmetic expressions over `Entry` values in `PostEntryExit`, which would directly absorb several Weak cases by letting the developer write the exact increment, and (b) a `returnExpression` clause for view functions, which would absorb several L4b cases by giving view-only functions a meaningful annotation target—are concrete directions identified by this analysis and discussed in §6.2.

5.4 RQ3: Comparison with Existing Detection Tools

We compare `IntentChecker` against three state-of-the-art tools on the 20 mitigated cases from RQ1: **ScType** (ICSE ’24), a static type-checker that assigns financial types (token unit, scaling factor) to variables and detects type inconsistencies; **GPTScan** (ICSE ’24), an LLM-based multi-step pipeline that matches functions against ten DeFi-specific vulnerability rules (price manipulation, slippage, etc.); and **NumScout**, a symbolic-execution engine that searches for five predefined numeric defect patterns (division-in-path, operator order, precision loss, exchange problem, profit opportunity). Each tool was executed three times on the same machine; `ScType` and `NumScout` produce deterministic output, while `GPTScan`’s LLM-based pipeline may vary across runs.

NumScout execution adjustments. To run `NumScout` over our `Web3Bugs` subset we applied two classes of adjustments, both external to the bugs

Table 11: Signal characterization of the four compared tools.

	IntentChecker	ScType	GPTScan	NumScout
Approach	Abstract interp. + Z3	Type inference	LLM + static	Symbolic exec.
Detection model	Specification-driven	Type-driven	Pattern-driven	Pattern-driven
Annotation	During/Post conds.	Financial types	None	None
Target class	Developer-specified	Type inconsistency	DeFi logic (10 rules)	Numeric (5 patterns)

under study. First, a source-level bug in NumScout’s semantic analyzer (`feature_detector/semantic.analysis.py`, in the `safeTransfer` handler) raised `Unbound-LocalError` on every contract reaching the relevant branch; we applied a minimal one-line fix (renaming an undefined local to the intended stack variable) so the tool could complete analysis. Second, NumScout consumes EVM bytecode and fails with “incomplete push instruction” errors when the bytecode contains unresolved library placeholders (`--$.$.--`), which are emitted when a contract calls a public/external library function that would normally be linked at deploy time. Two of our cases (51.H.02, `Swap.rampTargetPrice`; 77.H.01, `Exchange.calculateLiquidityTokenQtyForSingleAssetEntry`) use external libraries in this way. For these two cases only, we produced a second copy of the target file in which every public/external library function was rewritten as internal, causing the compiler to inline the library body into the calling contract’s bytecode rather than emit a placeholder. This transformation is semantically equivalent at the operation level—the arithmetic instructions NumScout inspects (`MUL`, `DIV`, `SUB`, etc.), the storage accesses, and the control flow are preserved—so the defect patterns NumScout targets are unchanged; only the call-site mechanism (`DELEGATECALL` vs. inlined code) differs. The modified files are kept separate from the originals used in the ScType and GPTScan experiments, and both modifications are restricted to the two cases noted above.

Detection model. A key distinction among these tools is how the set of detectable bugs is defined. GPTScan and NumScout operate on a pattern-driven model: each tool ships with a fixed catalog of vulnerability patterns (10 rules and 5 patterns, respectively), and only bugs matching those patterns can be detected. When a new class of numeric error emerges, the catalog must be updated by the tool developers. ScType is type-driven: it checks whether arithmetic operations preserve financial type consistency, which is more general than a fixed pattern catalog but still limited to errors expressible as type mismatches. IntentChecker is specification-driven: developers write `@During/@Post` conditions describing intended behavior, and the tool verifies the implementation against those conditions via abstract interpretation. This allows IntentChecker to target any numeric property expressible within its annotation grammar—including novel error patterns not yet cataloged by existing tools—but requires developer participation. The grammar’s expressibility ceiling was quantified in RQ2.

Finding 1: coverage landscape. Table 12 shows the per-case mitigation results. GPTScan reported findings on 11 of the 20 target files; however, every finding belongs to a different vulnerability class (price manipulation, slippage limit, etc.) than the actual numeric logic error. GPTScan’s rule set contains no rule for erroneous accounting or incorrect arithmetic formulas; consequently, the entire class of numeric logic errors falls outside its scope. This is not a deficiency of GPTScan—the tool was designed for a different threat model—but it demonstrates that a coverage gap exists: the bug class targeted by IntentChecker is not addressed by the most recent LLM-based smart-contract analysis tool.

NumScout mitigated the target bug in 9 of 20 cases. The nine mitigated cases concentrate on NumScout-origin samples (WANGMI, Nokon, BoostToken, HIT) and on Vader Protocol’s Utils.sol, whose division-heavy arithmetic matches NumScout’s division-in-path and operator-order patterns; Web3Bugs cases whose bugs lie outside these five patterns (e.g., CDP.update, Stream.updateStreamInternal, Liquidity-BasedTWAP.syncVaderPrice, RebaseProxy.mint) are not mitigated despite NumScout completing analysis. For 51_H_02 and 77_H_01, the reported NumScout entries are obtained on the library-inlined variant described earlier, and are marked with a dagger in Table 12.

Finding 2: complementarity with ScType. ScType provides the most direct comparison, as both tools target accounting errors and both require manual annotations (financial types for ScType, during/post conditions for IntentChecker). Among the seven overlapping cases, ScType mitigated four: calcAsymmetricShare, calcLiquidityUnits (both in Vader Protocol’s Utils.sol), transferFrom (Wrapped-IbbtcEth), and settleAccount (OptimisticLedger). IntentChecker mitigated all seven. The three cases that ScType could not mitigate (CDP.update, syncVaderPrice, _calcPrincipalWithdrawable) are not type-inconsistency errors but rather value-level errors: the arithmetic uses correctly-typed operands but produces a numerically wrong result. ScType’s type-driven model cannot express the property “this subtraction should yield the correct fee amount”; IntentChecker’s specification-driven model can. Conversely, ScType’s type inference operates without requiring the developer to know the expected output value, which makes it applicable even when the developer does not have a specific numeric property in mind. The two approaches are thus complementary: ScType catches type-level mismatches automatically, while IntentChecker catches value-level deviations when the developer can articulate the intended behavior.

Finding 3: analysis latency. Figure 6 reports per-case analysis times averaged over three runs. IntentChecker (mean 4.5 s) and ScType (mean 13.9 s) are both local, deterministic analyses that complete in seconds. GPTScan (mean 267 s) is approximately 59× slower due to its multi-step LLM pipeline, and NumScout (mean 772 s, median 517 s over 20 cases) is slower still, with several Web3Bugs cases running to the 1800 s global timeout set by NumScout’s symbolic-execution backend. The sub-10-second latency of IntentChecker makes it suitable for integration into CI/CD pipelines or interactive editor feedback loops, where developers expect near-instant responses.

Summary. Each tool occupies a distinct position in the detection landscape: pattern-driven tools (GPTScan, NumScout) offer fully automatic analysis but can only mitigate bugs within their predefined vulnerability catalogs; type-driven analysis

Table 12: Mitigation results on the 20 RQ1-mitigated cases. ✓ = mitigated (tool produced an actionable signal for the target bug), ✗ = not mitigated (tool was applicable but produced no signal), N/A = out of scope (no applicable rule or type file, or execution infeasible). GPTScan’s rule set does not target numeric logic errors, hence all cases are out of scope. NumScout entries marked [†] were evaluated on a semantically-equivalent variant in which external library functions were rewritten as internal to bypass NumScout’s library-placeholder limitation; see §5.4 for details.

Case	Contract.Function	IC	ScType	GPTScan	NumScout
WANGMI	WANGMI._transfer	✓	N/A	N/A	✓
Nokon	Nokon.buy	✓	N/A	N/A	✓
SwordCrowdsale	SwordCrowdsale.refundMoney	✓	N/A	N/A	✗
BoostToken (op)	BoostToken.sendETHToTeam	✓	N/A	N/A	✓
BoostToken (ind)	BoostToken.sendETHToTeam	✓	N/A	N/A	✓
HIT	HIT.getTokens	✓	N/A	N/A	✓
5_H.07	Utils.calcAsymmetricShare	✓	✓	N/A	✓
5_H.08	Utils.calcLiquidityUnits	✓	✓	N/A	✓
5_H.12	Pools.getAddedAmount	✓	N/A	N/A	✗
45_H.01	UToken.borrow	✓	N/A	N/A	✓
47_H.02	WrappedIbbtcEth.transferFrom	✓	✓	N/A	✗
51_H.02	SwapUtils.rampTargetPrice	✓	N/A	N/A	✗ [†]
56_H.02	CDP.update	✓	✗	N/A	✗
58_H.02	LpIssuer._chargeFees	✓	N/A	N/A	✗
60_H.01	OptimisticLedgerLib.settleAccount	✓	✓	N/A	✗
62_H.08	Stream.updateStreamInternal	✓	N/A	N/A	✗
70_H.10	LiquidityBasedTWAP.syncVaderPrice	✓	✗	N/A	✗
77_H.01	MathLib.calcLiqTokenQty	✓	N/A	N/A	✓ [†]
78_H.02	RebaseProxy.mint	✓	N/A	N/A	✗
101_H.01	LenderPool._calcPrincipalWithdrawable	✓	✗	N/A	✗

(ScType) generalizes beyond fixed patterns but is limited to errors expressible as type mismatches; and specification-driven analysis (IntentChecker) can mitigate any property within its annotation grammar but requires developer participation. The 20-case evaluation demonstrates that numeric logic errors—a class responsible for some of the highest-impact DeFi exploits—fall entirely outside the scope of pattern-driven tools and partially outside the reach of type-driven analysis, confirming the practical need for a specification-driven complement.

5.5 Threats to Validity

Internal validity. The intent and debug annotations used in RQ1 were authored by the researchers based on the audit report descriptions, not by the original contract developers. A different annotator could plausibly write a slightly different annotation for the same bug, and a sufficiently weak annotation could fail to surface a bug that a stronger annotation would catch. We mitigate this by deriving each annotation directly from the audit report’s stated root cause and recording the exact annotation alongside each case in the supplementary material, allowing independent re-evaluation.

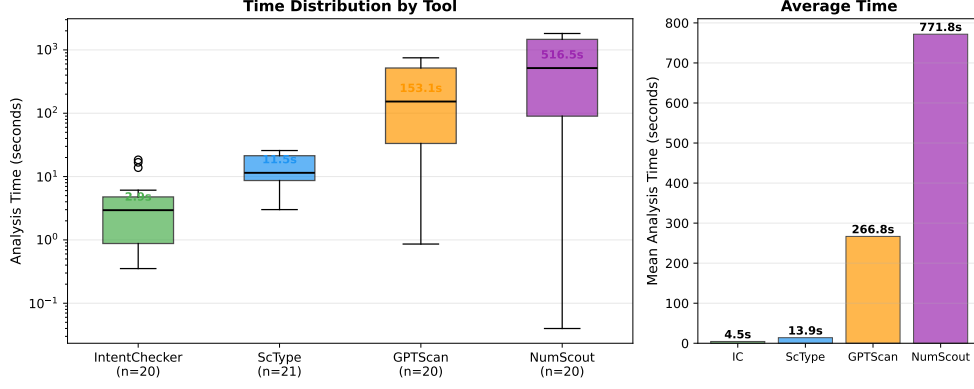


Fig. 6: Analysis time comparison across four tools (log scale). Left: box plot of per-case times over three runs. Right: mean analysis time. IntentChecker and ScType complete in seconds; GPTScan and NumScout each require minutes per case, with NumScout exceeding GPTScan on cases that exercise its symbolic-execution backend.

External validity. Our 89-case benchmark is drawn from Web3Bugs (Code4rena audit findings) and NumScout (BNB Smart Chain), which together cover a substantial portion of publicly known numeric logic errors but are not exhaustive. Findings may not generalize to numeric logic errors that arise in domains underrepresented by these two sources, such as zero-knowledge circuits or non-DeFi applications.

Construct validity. RQ1’s notion of “mitigation” relies on the validation engine producing a non-Satisfied verdict (typically Violated) for the annotated bug, rather than on a downstream measurement of developer behavior. A non-Satisfied verdict on its own does not guarantee that a developer reading it would correctly diagnose and fix the bug; assessing that downstream effect is an important direction for future work. In RQ3, GPTScan is marked as out of scope (N/A) rather than “not mitigated” because its rule set does not target numeric logic errors; this reflects a scope boundary, not a quality deficiency. The ScType comparison is limited to seven overlapping cases for which type annotation files exist; results on the remaining thirteen cases may differ. Two NumScout cases (51_H.02, 77_H.01) were evaluated on a variant of the target file in which public/external library functions were rewritten as internal to bypass NumScout’s inability to handle unresolved library placeholders in compiled bytecode (§5.4). Because this transformation only alters the call-site mechanism (inlined library code vs. DELEGATECALL) and preserves the arithmetic instructions, storage accesses, and control flow that NumScout’s defect patterns inspect, we regard the comparison as fair for the specific pattern-level signal NumScout produces; we nevertheless note the modification explicitly so that readers interpreting Table 12 can account for it.

6 Discussion

6.1 Annotation Authoring Burden and Future Automation

IntentChecker’s mitigation results in §5.2 are conditional on the developer supplying intent and debug annotations. This is by design—the entire approach is predicated on the developer making their intent explicit—but it nevertheless constitutes the tool’s most significant practical limitation. Two distinct kinds of effort are involved.

Effort 1: writing intent annotations for unfamiliar code. The intent model assumes that the developer can articulate, in the grammar of @During/@Post clauses, what the function is supposed to do. For code the developer wrote themselves a few minutes ago, this is unproblematic: the intent is still in the developer’s head. For code the developer is reviewing, modifying, or debugging without prior context, the intent must first be reconstructed from the surrounding code, comments, and external specifications, and only then can it be encoded as an annotation. This reconstruction step is not free, and it is exactly the step that the L5 (bug-awareness-required) limitation in §5.2 measures: when the correct intent is non-obvious, an annotation that captures it cannot be written without already knowing the bug.

Effort 2: supplying debug annotations for realistic execution paths. Even for code whose intent is clear, IntentChecker can only analyze the path that the developer’s debug annotations actually exercise. @StateVar, @LocalVar, and @IReturn require the developer to choose representative ranges; choosing ranges that miss the buggy region produces a false negative, while choosing ranges that are too wide invites widening and produces uninformative Warning verdicts. This is the standard cost of any annotation-based analysis and is well understood, but it remains a non-trivial barrier to adoption.

The circularity objection. A natural question is whether writing a correct intent annotation already presupposes knowing the bug—in which case the tool would only confirm what the developer already knows. The concern is legitimate, and our RQ2 analysis quantifies it precisely: 14 of the 34 intent-model limitation cases (L5) require bug awareness before the annotation can be written, confirming that for those cases, independent discovery through annotation alone is not realistic. However, this circularity is not unique to IntentChecker; it is inherent in any specification-based approach, from formal pre/post-conditions to Scribble annotations to test oracles. What differs across tools is how much of the specification burden falls on the developer. For the 20 mitigated cases in RQ1, the annotations were derivable from the developer’s own stated intent (e.g., “this fee should equal x ”, “this balance should decrease after withdrawal”) without requiring knowledge of the specific bug. The L5 cases, by contrast, identify the boundary where intent-driven annotation reaches its limit and external assistance—such as automated annotation suggestion—becomes necessary. This observation directly motivates the LLM-assisted workflow described below: rather than expecting the developer to write all annotations from scratch, a suggestion layer can propose candidate annotations that the developer accepts, rejects, or refines, shifting the task from generation to validation and partially breaking the circularity.

Future direction: LLM-assisted annotation suggestion and auto-repair. A natural way to lower both barriers is to combine IntentChecker with a large language

model that observes the same source the developer is editing. For intent annotations, the LLM could generate candidate `@During/@Post` clauses by reading the surrounding function, the contract’s NatSpec comments, and the developer’s recent commit messages, presenting the candidates as suggestions that the developer accepts or edits. For debug annotations, the LLM could propose representative ranges based on the function’s call sites elsewhere in the codebase or based on standard token decimals (e.g., 18 for most ERC20 tokens, 6 for USDC). Beyond annotation suggestion, an LLM that has access to IntentChecker’s Violated verdict and violated region could also propose code edits that move the violated region out of the reachable interval, effectively turning the awareness signal into a candidate fix. We deliberately leave both directions as future work; what matters for this paper is that they are natural extensions enabled by, rather than blocked by, the current design.

Developer study. This paper evaluates IntentChecker through structural analysis—what the tool can and cannot detect (RQ1), what the intent model can and cannot express (RQ2), and how the tool compares with existing detectors (RQ3)—rather than through a developer study measuring perceived usability or annotation intuitiveness. A controlled study with Solidity-experienced participants would answer a complementary set of questions: whether developers find the annotation syntax natural, how long annotation authoring takes in practice, and whether the tool’s feedback leads to faster bug discovery compared to existing debugging workflows. Designing such a study requires careful attention to participant recruitment (sufficient Solidity expertise) and ecological validity (realistic development tasks rather than isolated code snippets), and we defer it to future work as an independent evaluation axis that builds on the structural foundation established here.

6.2 Remaining Technical Limitations

In addition to the annotation authoring burden discussed above, three classes of technical limitations remain in the current implementation.

Multi-transaction invariants. IntentChecker’s analysis scope is multi-contract but single-transaction. A whole class of important numeric properties cannot be expressed within a single transaction because they relate quantities established by different transactions in the contract’s lifetime. The canonical example is the ERC20 conservation invariant `totalSupply == sum(balances)`: any single function execution may temporarily increment one side and not the other, but across the entire transaction history the two must agree. Another example is the price oracle manipulation pattern, where the buggy behavior only manifests when one transaction sets up state that a later transaction reads. Validating such properties would require tracking state across transactions and aggregating across all balance entries, both of which lie outside the current single-transaction interval-domain analysis. This is a known boundary of the development-time intent validation approach we have taken, and is more naturally addressed by post-hoc tools or formal verification frameworks that analyze entire contract lifecycles.

Non-arithmetic Yul and other unsupported Solidity constructs. The arithmetic-focused Yul subset described in §3.2(d) covers the cases where inline assembly is used as a faster substitute for high-precision integer arithmetic, which is by far the most common use case in DeFi libraries. However, hand-written Yul that uses memory and storage opcodes (mload/mstore, sload/sstore), control flow (if, for, switch), or function definitions falls outside the supported subset and is conservatively treated as $\top$. The same conservative treatment applies to a small number of Solidity constructs that the abstract interpreter does not model: `abi.decode` from raw bytes, `keccak256`, `address(this).balance` (whose value depends on the EVM-level execution context rather than on tracked state), and other built-ins whose results depend on opaque cryptographic or VM-level operations. Bugs whose buggy/correct distinction depends on the precise value produced by such constructs are correspondingly out of IntentChecker’s reach (these account for the L3 cases in Figure 5). Extending the Yul visitor to handle memory and storage opcodes and the abstract interpreter to handle a wider set of opaque builtins is a meaningful but bounded engineering task.

Soundness bounded by debug annotation scope. IntentChecker’s analysis is sound within the ranges declared by debug annotations: when the validation engine reports Satisfied, the declared intent provably holds for every combination of values in those ranges—this follows from the set-theoretic definition of the comparison primitive (§4.3.2) and the soundness results of §4.4.3 applied pointwise to each clause. The guarantee does not extend outside those ranges. A bug that only manifests at the boundary of a parameter’s representable values, but not in the range the developer chose for `@LocalVar`, will not appear in any verdict. This is an inherent characteristic of debugging tools—traditional debuggers also only execute the inputs the developer provides—and we encourage developers to use representative ranges and to iteratively refine them when Warning results draw attention to specific sub-intervals.

Value-level intent vs. logic-level intent. A broader observation from the limitation taxonomy is that IntentChecker’s intent model operates at the value level: it can express whether a variable’s value satisfies a numeric relationship (“this fee should be at least x ”, “this balance should decrease”), but it cannot express whether the right computation was performed in the first place. The 20 mitigated cases in RQ1 are precisely the cases where a value-level assertion suffices to distinguish buggy from correct execution. The L4 cases reveal the boundary: in L4a (10 cases), the correct expected value requires a computation that does not exist in the program; in L4b (8 cases), the function modifies no state variable, so there is no value to assert on. These are not failures of precision but of abstraction level—the developer’s true intent is about what the code should do (“split the swap at the crossing point”, “update total-Locked”), not about what value a variable should hold. Capturing such logic-level intent would require a fundamentally different formalism, such as temporal properties over execution traces or structural specifications of required code patterns, and lies outside the scope of a value-based interval-domain tool. Recognizing this boundary is important: IntentChecker is effective when the bug manifests as a wrong value, and complementary tools are needed when the bug manifests as a missing or misplaced operation.

6.3 Practical Annotation Guidelines

The annotation grammar (§4) defines what can be written; the question that remains is when a developer should write which annotation. The 20 mitigated cases and 14 L5 expressible cases reveal a recurring pattern: the most impactful annotations are not the ones that verify what the developer did write, but the ones that catch what the developer forgot to write. We distill three non-obvious guidelines from this observation.

Guideline 1: Enumerate all state variables a function should affect, not just those it does affect. Developers naturally reason about the code they wrote—the variables they explicitly assigned, the calls they explicitly made. Bugs of omission (L5a, 7 cases) arise precisely from this cognitive bias: a state variable that should have been updated was simply never mentioned. After writing a function, the developer should list every state variable whose value the function is expected to change and add at least a directional annotation (`@Post var(Entry; Exit)` or `@Post changed(var, true)`) for each. In our data, this single practice—checking what should change rather than what does change—would have caught 7 of the 14 L5 expressible cases and 5 of the 8 mitigated `@Post` cases, all without requiring any knowledge of the specific bug.

Guideline 2: When two state variables should move together, annotate their relationship, not each one in isolation. Several benchmark cases involve accounting pairs (e.g., a per-user balance and a global total, a deposit ledger and a reward tracker) that should stay in sync. Annotating each variable independently catches only missing-update bugs; annotating their relationship (e.g., `@Post globalTotal(Entry - Exit) += userBalance(Entry - Exit)`) catches bugs where both variables are updated but by inconsistent amounts. This pattern recurs in 4 of the 20 mitigated cases (all Erroneous Accounting) and is the dominant form in L4d (invariant-masked), where independent annotations on each variable succeed but the cross-variable invariant fails.

Guideline 3: For multi-step arithmetic, annotate the result against the mathematically ideal expression, not the code as written. Solidity’s integer division truncates at every intermediate step, and the resulting precision loss depends on the order of operations. Code that reads `amount.div(12).mul(5)` may look correct, but the developer’s actual intent is the mathematical value $\lfloor \text{amount} \times 5 / 12 \rfloor$. Writing the intent annotation in the ideal form—`@During result += amount * 5 / 12`—causes `IntentChecker` to compare the truncated code path against the non-truncated reference, surfacing the discrepancy automatically. This pattern accounts for 7 of the 20 mitigated cases (Div In Path, Operator Order Issue, Exchange Problem, and Precision Loss Trend), all of which were caught by annotations that express the mathematically correct expression rather than restating what the code already does.

7 Related Works

7.1 Numeric Logic Error Detection in Solidity

Logic error detection focuses on identifying discrepancies between developer intentions and actual program behavior in smart contracts.

(Eshghie et al., 2024) addresses business logic violations in smart contracts by using Dynamic Condition Response (DCR) graphs to model expected behaviors including

temporal constraints, role-based access control, and inter-function dependencies. The system monitors runtime transactions against user-provided DCR models and their function mappings, generating violation reports when execution activities deviate from the specified business logic patterns.

(Stephens et al., 2021) verifies temporal properties of smart contracts by accepting user-specified Linear Temporal Logic (LTL) formulas and attack models to check both safety and liveness properties. The system not only detects violations but also generates concrete attack scenarios that demonstrate how the temporal properties can be violated, providing developers with actionable insights into potential logic errors.

(Wei et al., 2024) enables developers to specify function and address behaviors using the ConSol syntax alongside their Solidity code, then verifies whether the implementation adheres to these behavioral specifications and generates behavior-compliant Solidity programs without annotations.

(Wang et al., 2024b) leverages multimodal learning to automatically infer invariants and detect machine un-auditable bugs (MUBs) (Zhang et al., 2023) by analyzing both source code and natural language artifacts including comments, function signatures, and documented transaction scenarios. The system employs a fine-tuned model with Tier of Thought (ToT) prompting that progressively predicts transaction contexts, generates and ranks invariants, and ultimately identifies vulnerabilities through three-tier reasoning.

(Zhang, 2024) proposes a novel type system for Solidity variables defined as tuples of (Financial Type, Scaling Factor, Token Unit, Address) to detect accounting errors through type checking. The system reinterprets Solidity code based on detection patterns such as incorrectly adding balances with fees, enabling systematic identification of numeric logic errors that arise from mismatched financial operations.

Furthermore, there are also approaches that address multiple transaction logic errors such as price manipulation (Wu et al., 2023; Wang et al., 2024a; Xi et al., 2024; Arora et al., 2024; Kong et al., 2023) and flash loan attacks (Li et al., 2025; Ramezany et al., 2023).

The numeric logic error detectors above and IntentChecker occupy distinct points along the development–auditing timeline rather than competing on the same axis. NumScout, GPTScan, SmartInv, and the type-system approach of Zhang (2024) are all designed to run over completed code and recover developer intent indirectly through pattern catalogs, language-model inference, or financial type assignments. IntentChecker is positioned earlier in time and lets the developer declare intent explicitly through annotations evaluated inside the editor. Because the two approaches see different inputs (code only vs. code with declared intent) and run in different workflow stages (post-hoc audit vs. edit-time validation), their coverage is necessarily complementary, and §5.4 characterizes that complementarity rather than arguing numerical superiority.

7.2 Specification-based Smart Contract Verification

Specification-based verification approaches focus on verifying general properties and correctness requirements of smart contracts rather than specific developer intentions.

([Consensys Diligence, 2025c](#)) provides a framework that transforms developer annotations into Solidity assertion code, enabling testing with existing tools like MythX ([Consensys Diligence, 2025b](#)) and Mythril ([Consensys Diligence, 2025a](#)). The framework supports multiple annotation types including `if_succeed` (conditions for successful function returns), `invariant` (contract-level properties that must always hold), `if_updated` (conditions checked when state variables change), and `assert` (validations at specific execution points), making formal specifications accessible through familiar testing pipelines.

([Wesley et al., 2022](#)) builds upon Scribble ([Consensys Diligence, 2025c](#)) to verify whether Solidity code satisfies specifications written in the Scribble language, leveraging C-based verification tools like SeaHorn ([Gurfinkel et al., 2015](#)), libFuzzer ([LLVM, 2025](#)), and KLEE ([Cadar et al., 2008](#)) by transforming smart contracts into LLVM-IR based harnesses. The framework abstracts the 2^{160} possible user addresses into a small set of representative users (implicit, transient, and explicit), enabling verification of properties that must hold for arbitrary numbers of users, such as accounting properties like “the contract balance equals the sum of all user deposits.”

([Nguyen et al., 2023](#)) verifies Solidity code using two types of annotations: specification annotations (loop invariants, call invariants, and call modifies) that developers declare as assumptions, and verification annotations (`ensures`, `reverts_if`, and contract invariants) that the tool checks against the implementation. The system assumes the truth of specification annotations and uses them to verify whether the code satisfies function post-conditions, correctly handles revert conditions, and maintains contract invariants throughout execution.

([Bartoletti et al., 2024](#)) focuses on verifying liquidity properties of smart contracts through formal methods by converting both the contract code and user-defined liquidity properties into SMT constraints. The system then checks whether the contract satisfies these properties using SMT solvers, generating counterexamples when violations are detected to help developers understand potential liquidity issues.

While specification-based approaches formalise contract-level properties in dedicated specification languages—Scribble’s annotation language, Idealist’s loop and call invariants, Solvent’s SMT-encoded liquidity properties—and discharge verification obligations over completed code, IntentChecker’s intent model occupies a different point in the design space with three distinguishing characteristics. First, its annotation language targets value-level numeric intent on individual variables at specific program points, not contract-level invariants or temporal properties. Second, its denotational semantics (§4.3.2) maps each clause form to a tri-valued validation outcome over the abstract states produced by interval-domain analysis, giving the annotation language a formal foundation comparable in rigor to the specification languages above while remaining cheap enough to evaluate at edit time. Third, because IntentChecker performs interval-domain analysis that tracks actual value ranges through library function calls (e.g., SafeMath operations) rather than treating libraries as black boxes, it can validate intent over step-by-step numeric computations rather than only over post-condition-style abstract properties. The two approaches are therefore complementary in workflow stage (edit-time vs. audit/verification-time), in granularity (value-level vs. contract-level), and in the specification style they support.

7.3 Tools for Strengthening Security During Development Phase

Development-phase security tools integrate vulnerability detection directly into the coding workflow, enabling developers to identify and address security issues as they write code rather than in post-development analysis. (Ren et al., 2021) presents an integrated development environment for smart contract development that combines natural language model-based code autocompletion with security validation. The platform integrates existing static analysis tools including Oyente (Luu et al., 2016), Mythril (Consensys Diligence, 2025a), and Securify (Tsankov et al., 2018) to provide comprehensive security reports during development workflow.

(Protofire, 2025) provides a Solidity linter that detects predefined vulnerability patterns including low-level calls, reentrancy, and deprecated function usage during development. The tool enforces security best practices by checking for patterns such as proper visibility declarations, send result validation, and secure compiler version usage.

While these development-phase tools rely on predefined detection patterns for known vulnerabilities, IntentChecker enables developers to specify their intended numeric behaviors through annotations and validates whether the actual program execution matches these specified intentions. IntentChecker also extends SolQDebug’s analysis scope beyond a single contract to a multi-contract single-transaction scope that includes inheritance, libraries, interfaces, and an arithmetic Yul subset, which is what makes intent validation feasible on realistic DeFi code.

8 Conclusion

We presented IntentChecker, a development-time intent validation tool that extends SolQDebug with (i) an intent model—two annotation types (@During/@Post) equipped with a formal denotational semantics that maps each clause to a tri-valued outcome against abstract states, (ii) a validation engine whose interval comparison primitive is proved sound against that semantics and cross-validated against brute-force ground truth, producing graded Satisfied/Warning/Violated signals with separately computed risk scores and violated regions, and (iii) a scope extension to multi-contract single-transaction analysis covering inheritance, libraries, interfaces, and an arithmetic Yul subset.

On 75 effective benchmark instances (89 collected, 14 excluded) from Web3Bugs and NumScout, IntentChecker surfaced 20 bugs via Violated verdicts at edit-time speeds (mean 3.69s per case). RQ2’s expressibility analysis showed that 14 of the 34 intent-model limitation cases are expressible under bug awareness, while 20 represent a hard grammar ceiling—identifying concrete extension points for the intent model. IntentChecker’s positioning relative to existing detection tools is complementary: it fills the edit-time gap that auditing-time tools structurally cannot occupy.

The primary practical limitation is the annotation authoring burden, which future work can address through LLM-assisted annotation suggestion (lowering the L5 bug-awareness ceiling) and auto-repair from violated regions. On the analyzer side,

extending Yul support and multi-transaction reasoning would close the remaining technical limitations. More broadly, this work adds “edit-time intent validator” as a third category alongside pattern-based detectors and formal verifiers in the design space for numeric logic error tools.

References

- Arora, S., et al.: SecPLF: Secure protocols for loanable funds against oracle manipulation attacks. In: Proceedings of the 19th ACM Asia Conference on Computer and Communications Security (ASIACCS), pp. 1394–1405 (2024). <https://doi.org/10.1145/3634737.3637681>
- Bartoletti, M., et al.: Solvent: liquidity verification of smart contracts. In: International Conference on Integrated Formal Methods (iFM), pp. 256–266 (2024). https://doi.org/10.1007/978-3-031-76554-4_14
- Cadar, C., et al.: KLEE: unassisted and automatic generation of high-coverage tests for complex systems programs. In: OSDI, pp. 209–224 (2008). <https://doi.org/10.5555/1855741.1855756>
- Chaliasos, S., et al.: Smart contract and defi security tools: Do they meet the needs of practitioners?. In: Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3623302>
- Chen, J., et al.: NumScout: Unveiling Numerical Defects in Smart Contracts using LLM-Pruning Symbolic Execution. IEEE Transactions on Software Engineering (2025). <https://doi.org/10.1109/TSE.2025.3555622>
- Consensys Diligence: Mythril. <https://github.com/ConsenSysDiligence/mythril> (2025). Accessed December 2025
- Consensys Diligence: MythX. <https://mythx.io/> (2025). Accessed December 2025
- Consensys Diligence: Scribble. <https://diligence.security/scribble/> (2025). Accessed January 2026
- Eshghie, M., et al.: Highguard: Cross-chain business logic monitoring of smart contracts. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 2378–2381 (2024). <https://doi.org/10.1145/3691620.3695356>
- Gurfinkel, A., et al.: The SeaHorn verification framework. In: International Conference on Computer Aided Verification, pp. 343–361 (2015). https://doi.org/10.1007/978-3-319-21690-4_20
- Jeon, I., et al.: SolQDebug: Debug Solidity Quickly for Interactive Immediacy in Smart Contract Development (2025). <https://doi.org/10.21203/rs.3.rs-8077153/v1>
- Kong, Q., et al.: Defitainter: Detecting price manipulation vulnerabilities in defi protocols. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), pp. 1144–1156 (2023). <https://doi.org/10.1145/3597926.3598124>

- Li, X., et al.: Penetrating the hostile: Detecting defi protocol exploits through cross-contract analysis. *IEEE Transactions on Information Forensics and Security* 20, 11759–11774 (2025). <https://doi.org/10.1109/TIFS.2025.3626979>
- LLVM: libFuzzer. <https://lvm.org/docs/LibFuzzer.html> (2025). Accessed December 2025
- Luu, L., et al.: Making smart contracts smarter. In: *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 254–269 (2016). <https://doi.org/10.1145/2976749.2978309>
- Nguyen, T.D., et al.: An idealist’s approach for smart contract correctness. In: *International Conference on Formal Engineering Methods*, pp. 11–28 (2023). https://doi.org/10.1007/978-981-99-7584-6_2
- Protofire: Solhint. <https://github.com/protofire/solhint> (2025). Accessed January 2026
- Ramezany, S., Setthawong, R., Setthawong, P.: Midnight: An Efficient Event-driven EVM Transaction Security Monitoring Approach For Flash Loan Detection. In: *Proceedings of the 2023 20th International Joint Conference on Computer Science and Software Engineering (JCSSE)*, pp. 237–241 (2023). <https://doi.org/10.1109/JCSSE58229.2023.10201970>
- Ren, M., et al.: Making smart contract development more secure and easier. In: *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, pp. 1360–1370 (2021). <https://doi.org/10.1145/3468264.3473929>
- Soud, M., Liebel, G., Hamdaqa, M.: A fly in the ointment: an empirical study on the characteristics of Ethereum smart contract code weaknesses. *Empirical Software Engineering* 29(1), 13 (2024). <https://doi.org/10.1007/s10664-023-10398-5>
- Stephens, J., et al.: SmartPulse: automated checking of temporal properties in smart contracts. In: *Proceedings of the 2021 IEEE Symposium on Security and Privacy (SP)*, pp. 555–571 (2021). <https://doi.org/10.1109/SP40001.2021.00085>
- Sun, Y., et al.: Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In: *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3639117>
- Tsankov, P., et al.: Securify: Practical security analysis of smart contracts. In: *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pp. 67–82 (2018). <https://doi.org/10.1145/3243734.3243780>
- Wang, D., et al.: Defiguard: A price manipulation detection service in defi using graph neural networks. *IEEE Transactions on Services Computing* (2024). <https://doi.org/10.1109/TSC.2024.3489439>
- Wang, S.J., Pei, K., Yang, J.: Smartinv: Multimodal learning for smart contract invariant inference. In: *Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP)*, pp. 2217–2235 (2024). <https://doi.org/10.1109/SP54263.2024.00126>
- Wei, G., et al.: Consolidating Smart Contracts with Behavioral Contracts. *Proceedings of the ACM on Programming Languages* 8(PLDI), 965–989 (2024).

- <https://doi.org/10.1145/3656416>
- Wesley, S., et al.: Verifying Solidity smart contracts via communication abstraction in SmartACE. In: International Conference on Verification, Model Checking, and Abstract Interpretation, pp. 425–449 (2022). https://doi.org/10.1007/978-3-030-94583-1_21
- Wu, S., et al.: Defiranger: Detecting defi price manipulation attacks. IEEE Transactions on Dependable and Secure Computing 21(4), 4147–4161 (2023). <https://doi.org/10.1109/TDSC.2023.3346888>
- Xi, R., Wang, Z., Pattabiraman, K.: POMABuster: Detecting Price Oracle Manipulation Attacks in Decentralized Finance. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 3923–3942 (2024). <https://doi.org/10.1109/SP54263.2024.00257>
- Zhang, B.: Towards finding accounting errors in smart contracts. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE), pp. 1–13 (2024). <https://doi.org/10.1145/3597503.3639128>
- Zhang, W., et al.: Nyx: Detecting exploitable front-running vulnerabilities in smart contracts. In: Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP), pp. 2198–2216 (2024). <https://doi.org/10.1109/SP54263.2024.00146>
- Yang, S., et al.: Hyperion: Unveiling DApp Inconsistencies Using LLM and Dataflow-Guided Symbolic Execution. In: Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), pp. 2125–2137 (2025). <https://doi.org/10.1109/ICSE55347.2025.00015>
- Zhang, Z., et al.: Demystifying exploitable bugs in smart contracts. In: Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE), pp. 615–627 (2023). <https://doi.org/10.1109/ICSE48619.2023.00061>
- Zheng, Z., et al.: Dappscan: building large-scale datasets for smart contract weaknesses in dApp projects. IEEE Transactions on Software Engineering 50(6), 1360–1373 (2024). <https://doi.org/10.1109/TSE.2024.3383422>
- Zhou, L., et al.: Sok: Decentralized finance (defi) attacks. In: Proceedings of the 2023 IEEE Symposium on Security and Privacy (SP), pp. 2444–2461 (2023). <https://doi.org/10.1109/SP46215.2023.10179435>